

国立大学法人

電気通信大学

The University of Electro-Communications

Pham Laboratory
Integrated circuit design laboratory

Tutorial

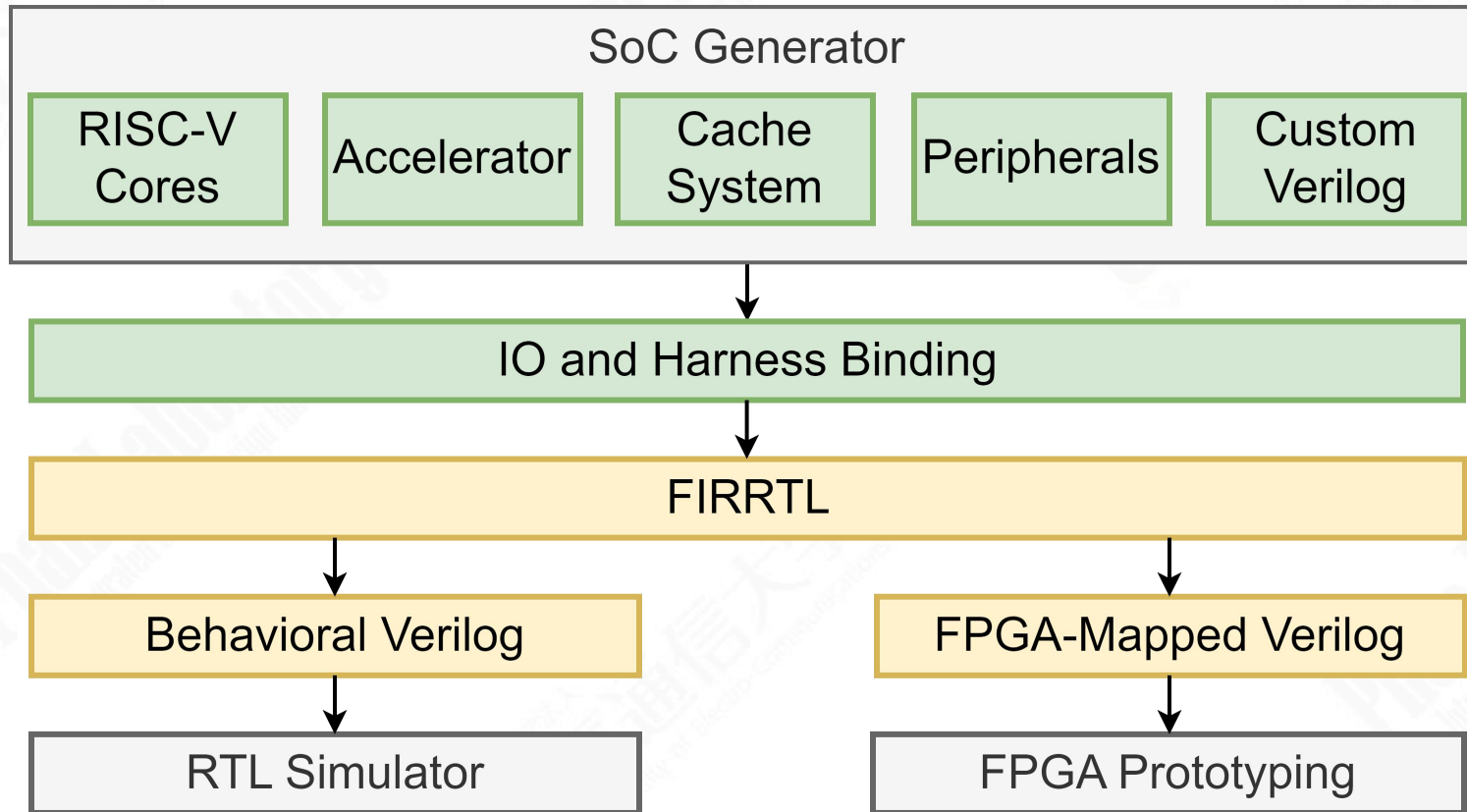
Outline

1. Framework organization
2. SoC Generation
3. FPGA Prototyping
4. Git clone and prepare
5. Make the system
6. Prepare SDCard
7. Using IDE
8. Modifying system by Scala config

Outline

1. Framework organization
2. SoC Generation
3. FPGA Prototyping
4. Git clone and prepare
5. Make the system
6. Prepare SDCard
7. Using IDE
8. Modifying system by Scala config

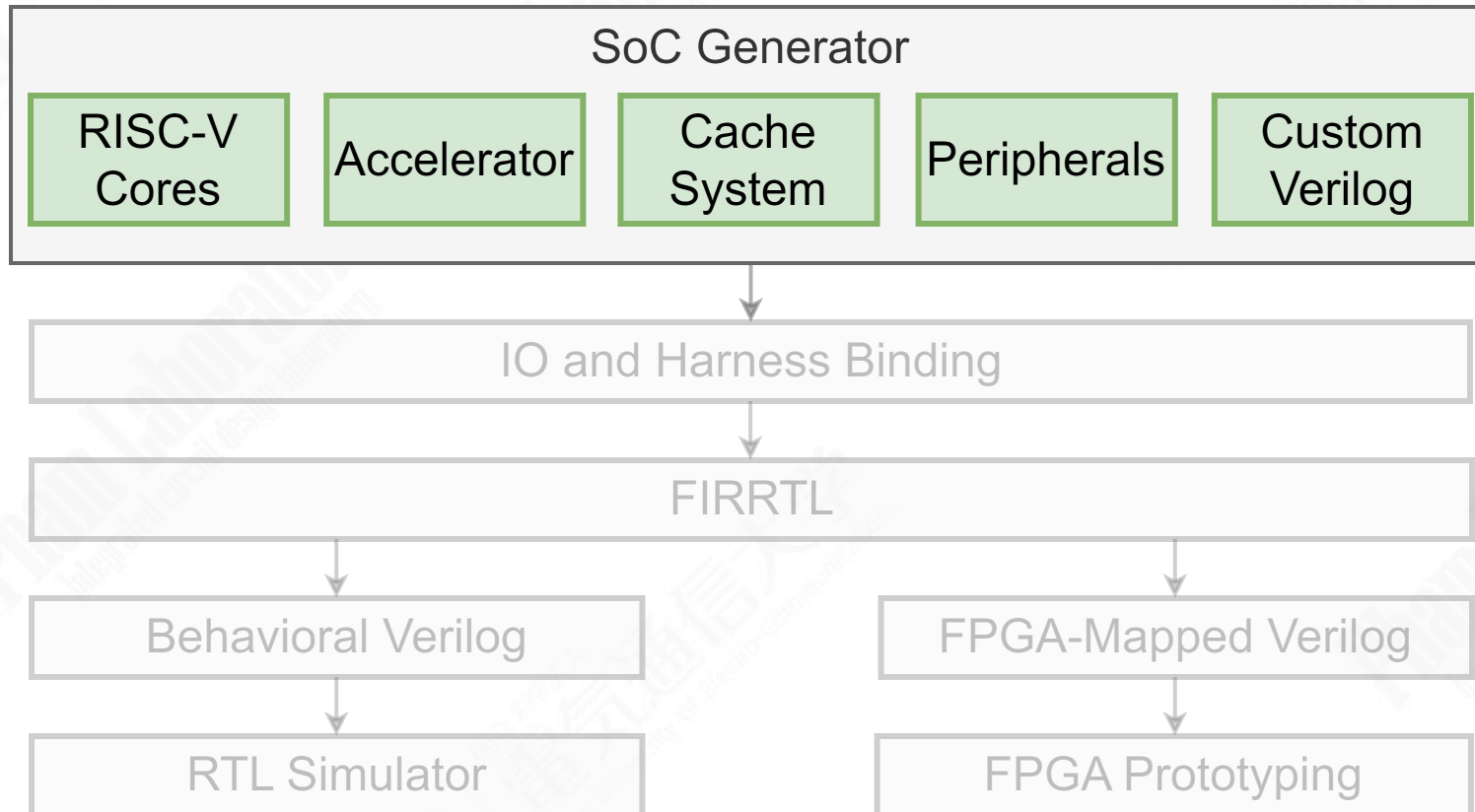
1. Framework Organization (1/7) Working flow



What is Chipyard?

1. An organized framework for various SoC design tools.
2. An IP library of open-source RISC-V SoC components.
3. A methodology for agile SoC architecture design, exploration, and evaluation.

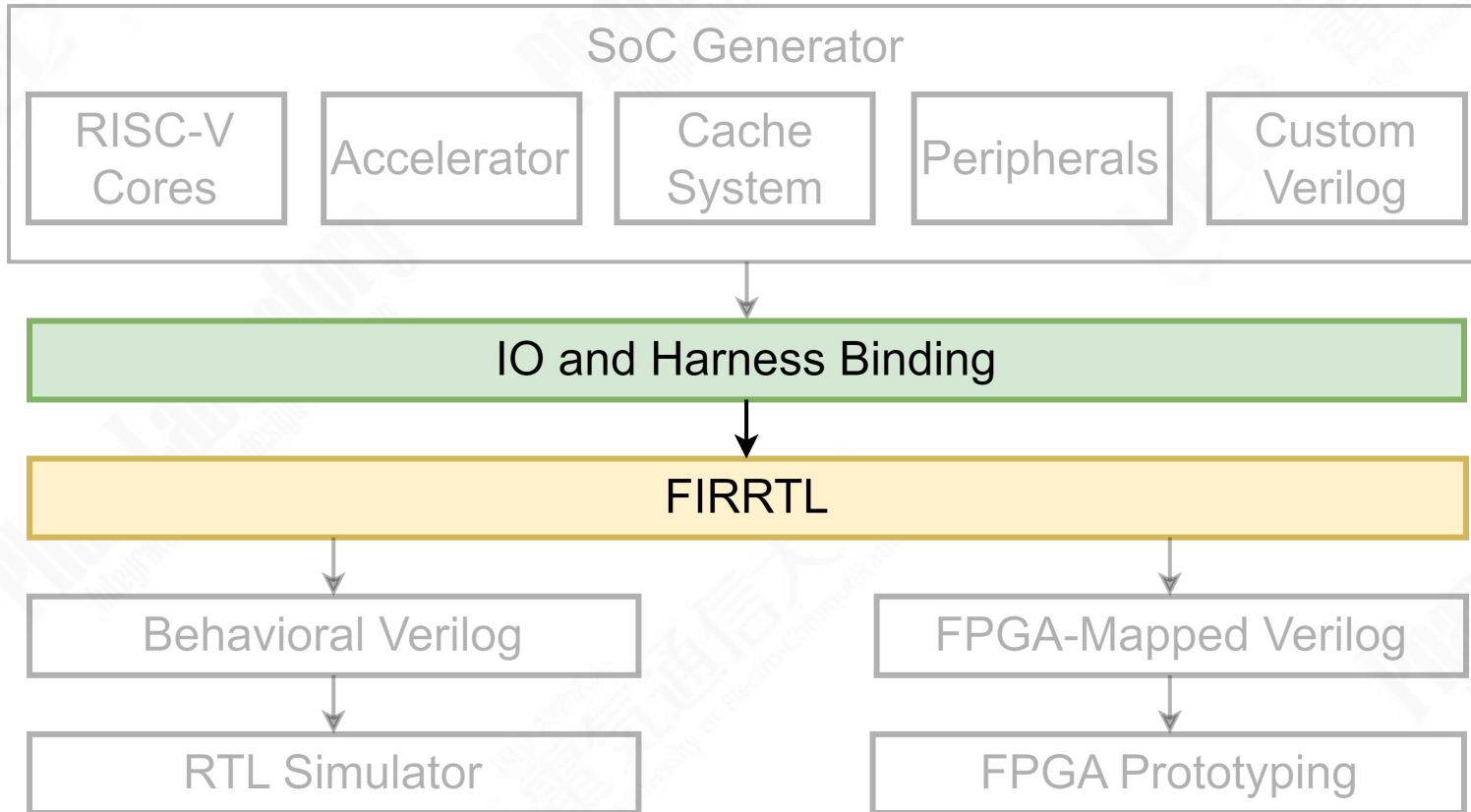
1. Framework Organization (2/7) Working flow



A set of IPs (described by Chisel or Custom Verilog) that are necessary for System on Chip (SoC), including:

- RISC-V Cores (Rocket, BOOM, Ibex, CVA6).
- Custom hardware (RoCC or Peripheral).
- Memory system.
- Peripherals (GPIOs, I2C, SPI, etc).
- Custom Verilog IPs.

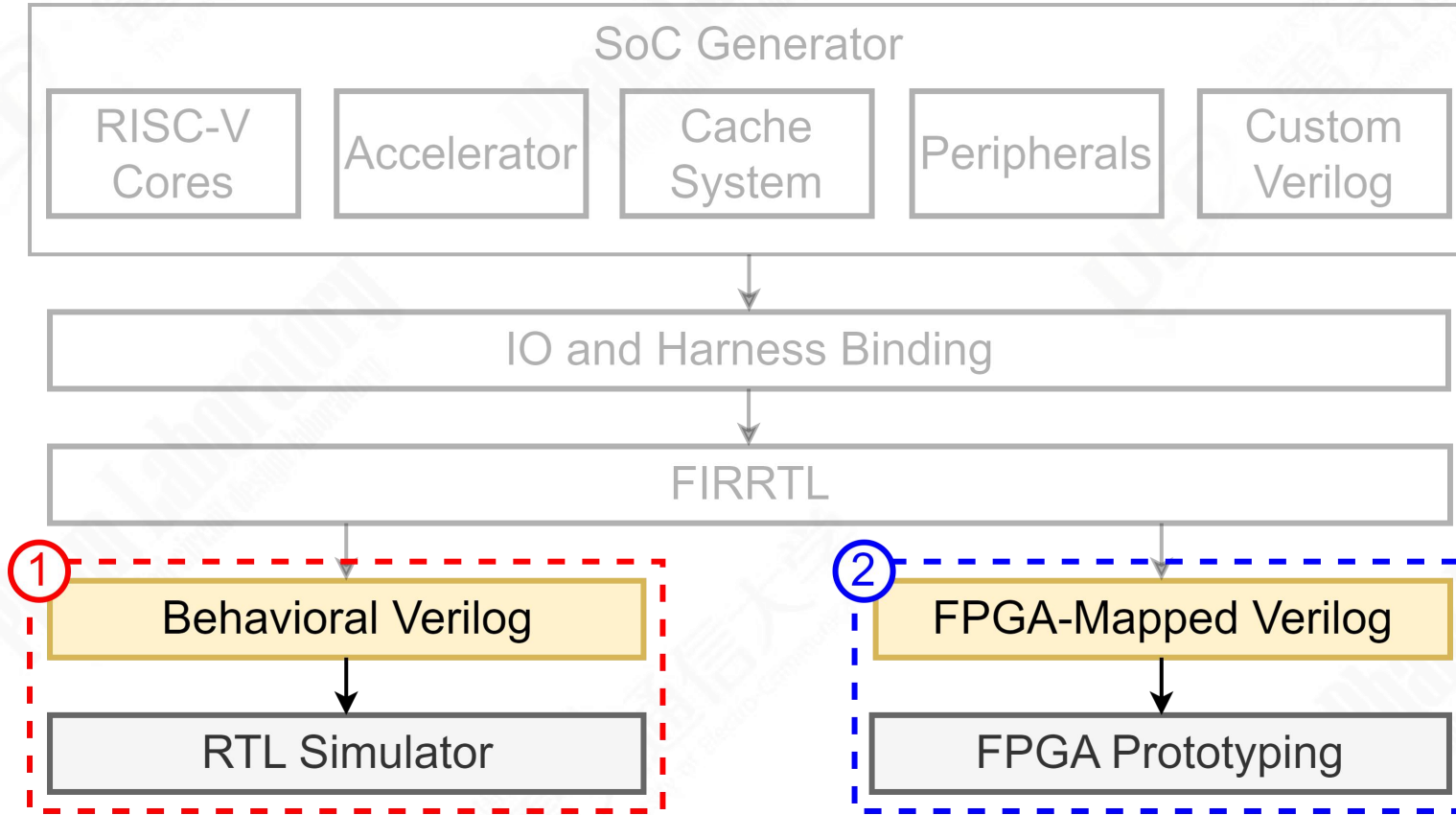
1. Framework Organization (3/7) Working flow



IO and Harness Binding bind the generated SoC to the corresponding IO Cells and IO Pins.

FIRRTL compiles Chisel to corresponding Verilog.

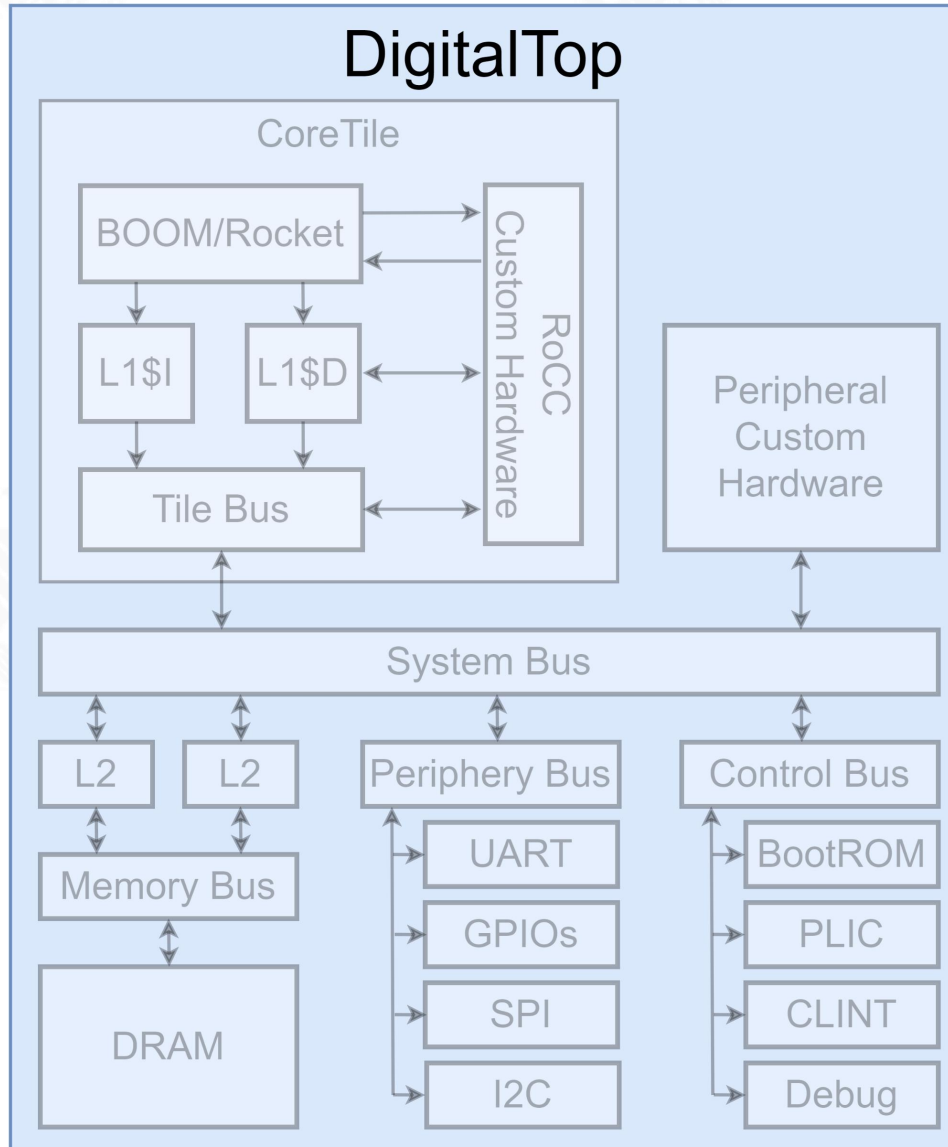
1. Framework Organization (4/7) Working flow



1. Generated Verilog sources of the SoC, peripherals' behaviors are simulated by input C sources.
2. Generated Verilog sources of the SoC and peripherals are mapped with the FPGA prototype. The generated bitstream file could be deployed into real FPGA platform.

1. Framework Organization (5/7) Organization

A Chipyard project is organized in 3 levels:



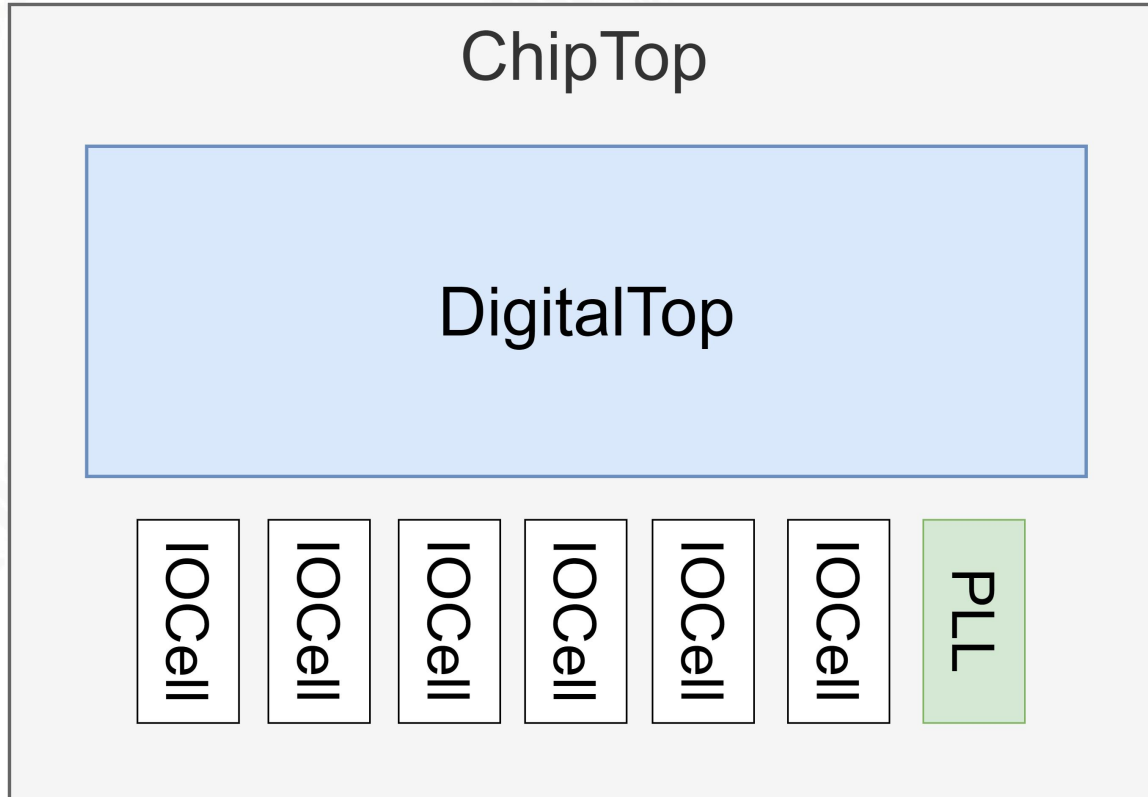
DigitalTop

At this level, the SoC architecture is defined with the customized components:

- Core types.
- Cache size/cache organization.
- Bus system.
- Custom hardware (RoCC or Peripheral).
- Peripherals.
- Control components (BootROM, PLIC, CLINT, Debug).

1. Framework Organization (6/7) Organization

A Chipyard project is organized in 3 levels:



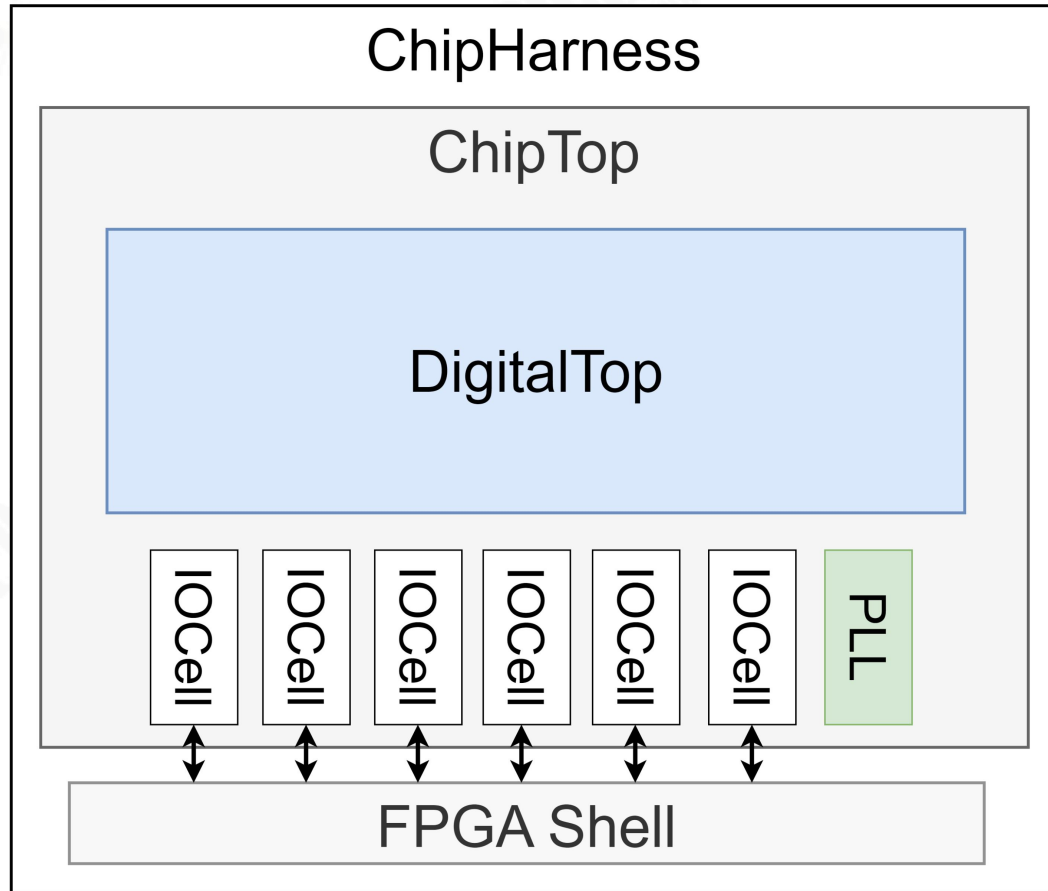
ChipTop

At this level, the DigitalTop I/O will be connected with the corresponding IOCell, such as:

- Analog.
- Digital Input/Output/Bi-directions.
- Clock.

1. Framework Organization (7/7) Organization

A Chipyard project is organized in 3 levels:



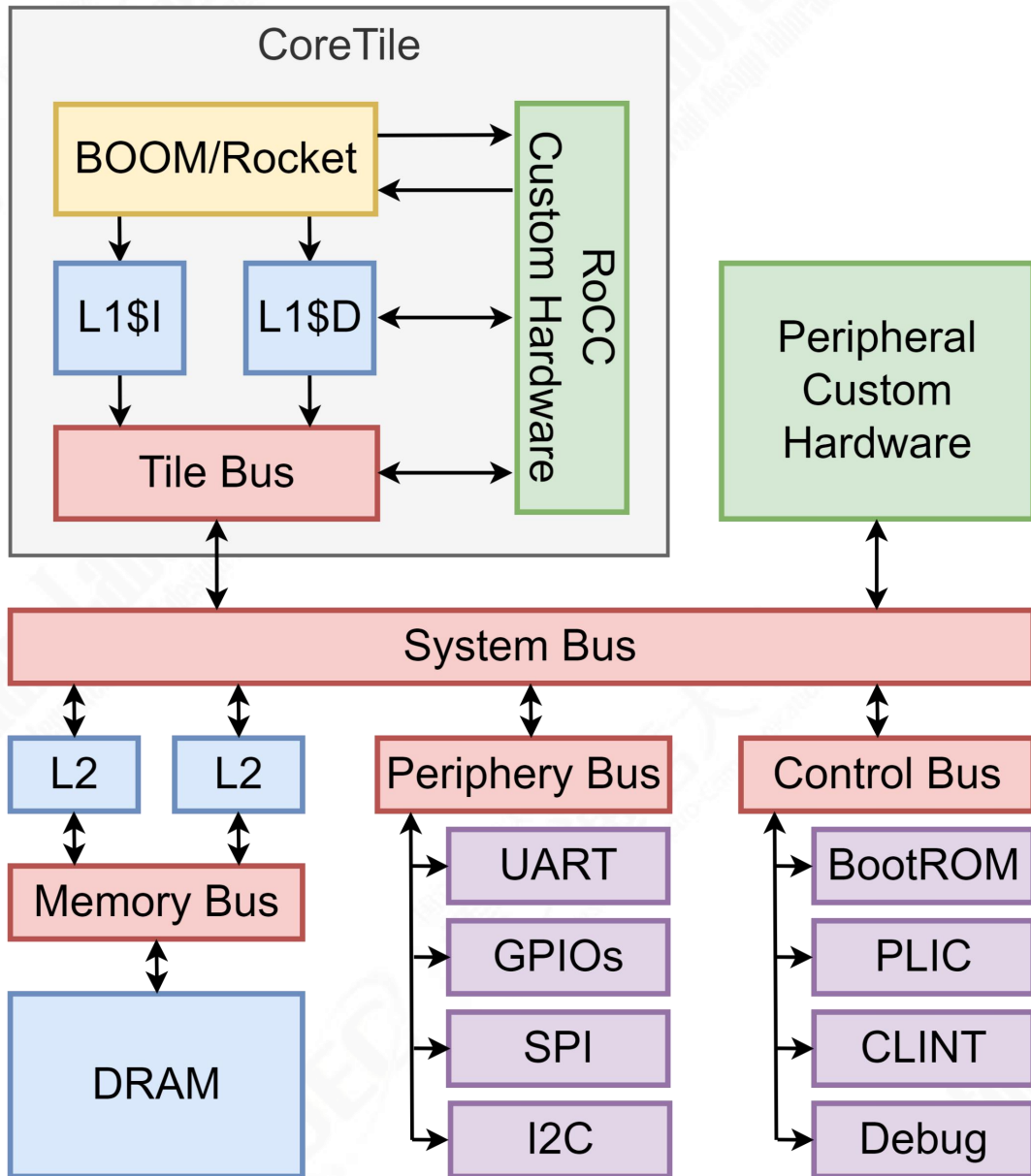
ChipHarness

At this level, the IOCells are connected with the physical IO of the deployed FPGA platform. Including timing and physical constraints.

Outline

1. Framework organization
2. **SoC Generation**
3. FPGA Prototyping
4. Git clone and prepare
5. Make the system
6. Prepare SDCard
7. Using IDE
8. Modifying system by Scala config

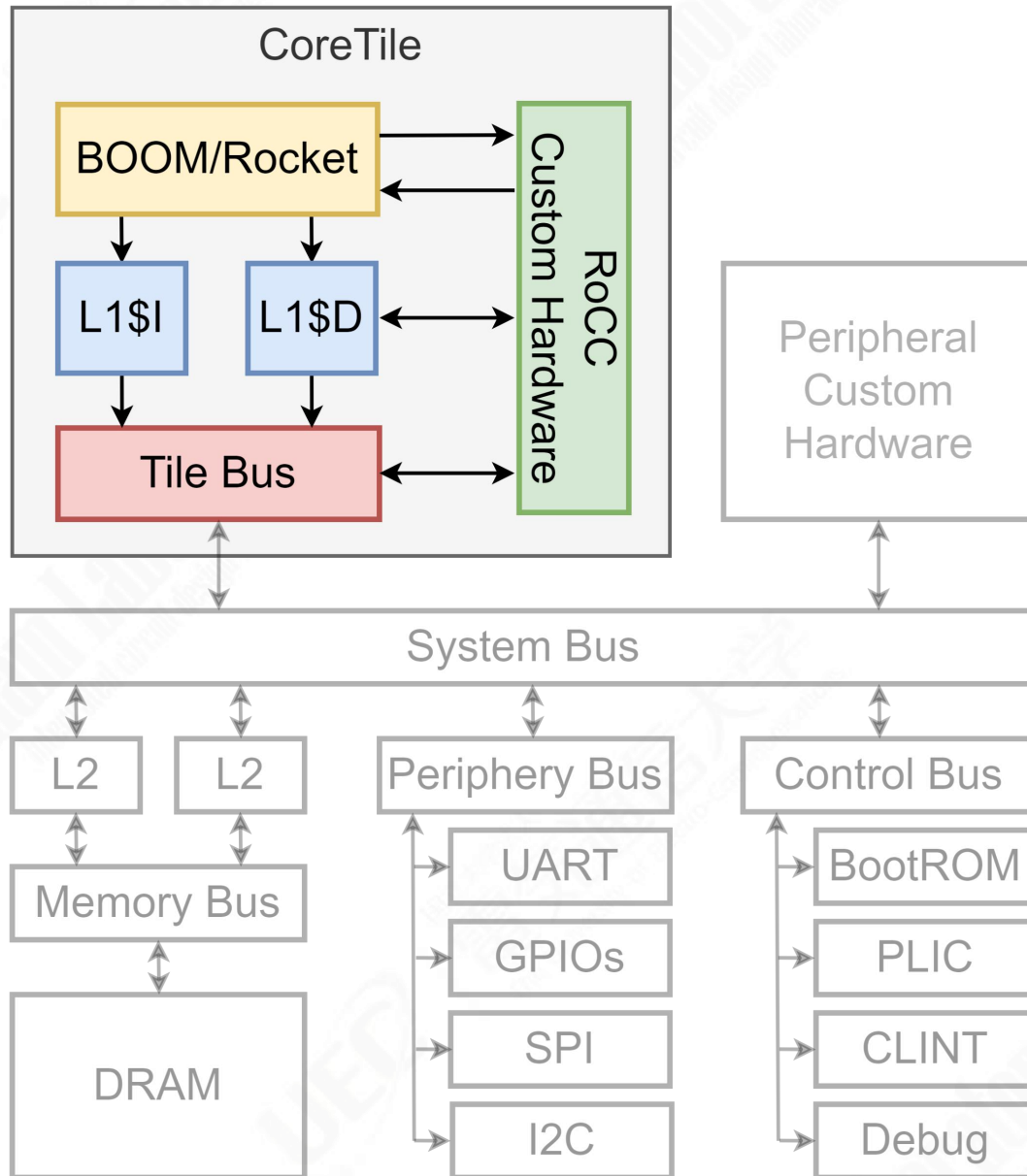
2. SoC Generation (1/11) SoC Architecture



In Chipyard framework, a SoC includes 5 components:

1. RISC-V Core (or Tile).
2. Memory system.
3. Bus system.
4. Custom hardware.
5. Peripherals.

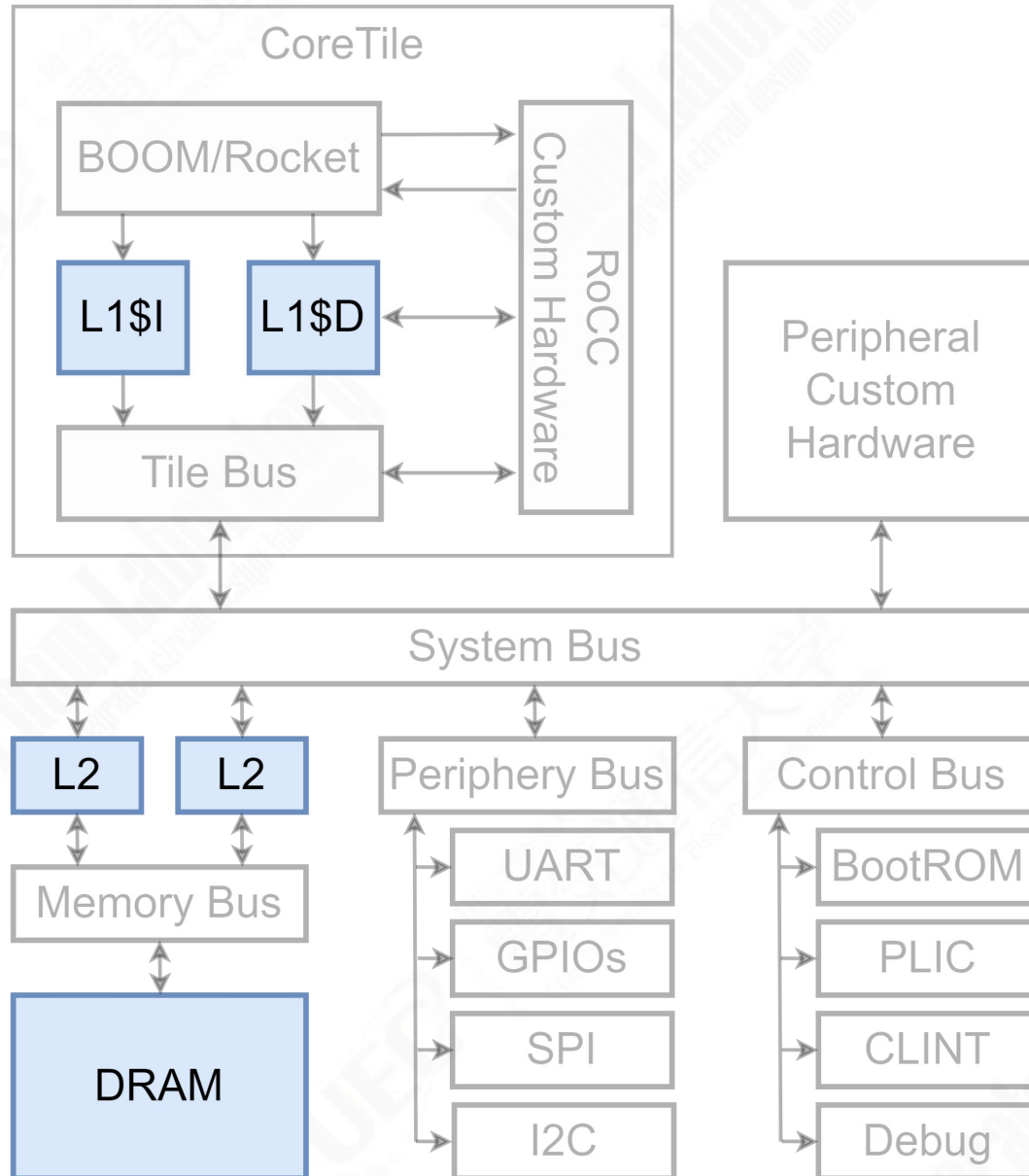
2. SoC Generation (2/11) SoC Architecture



Tile and Core:

- Each Tile contains a RISC-V core and private caches.
- Varieties of Cores are supported (Rocket, BOOM, Ibex, CVA6).
- The interface supports integrating your own customized RISC-V core.

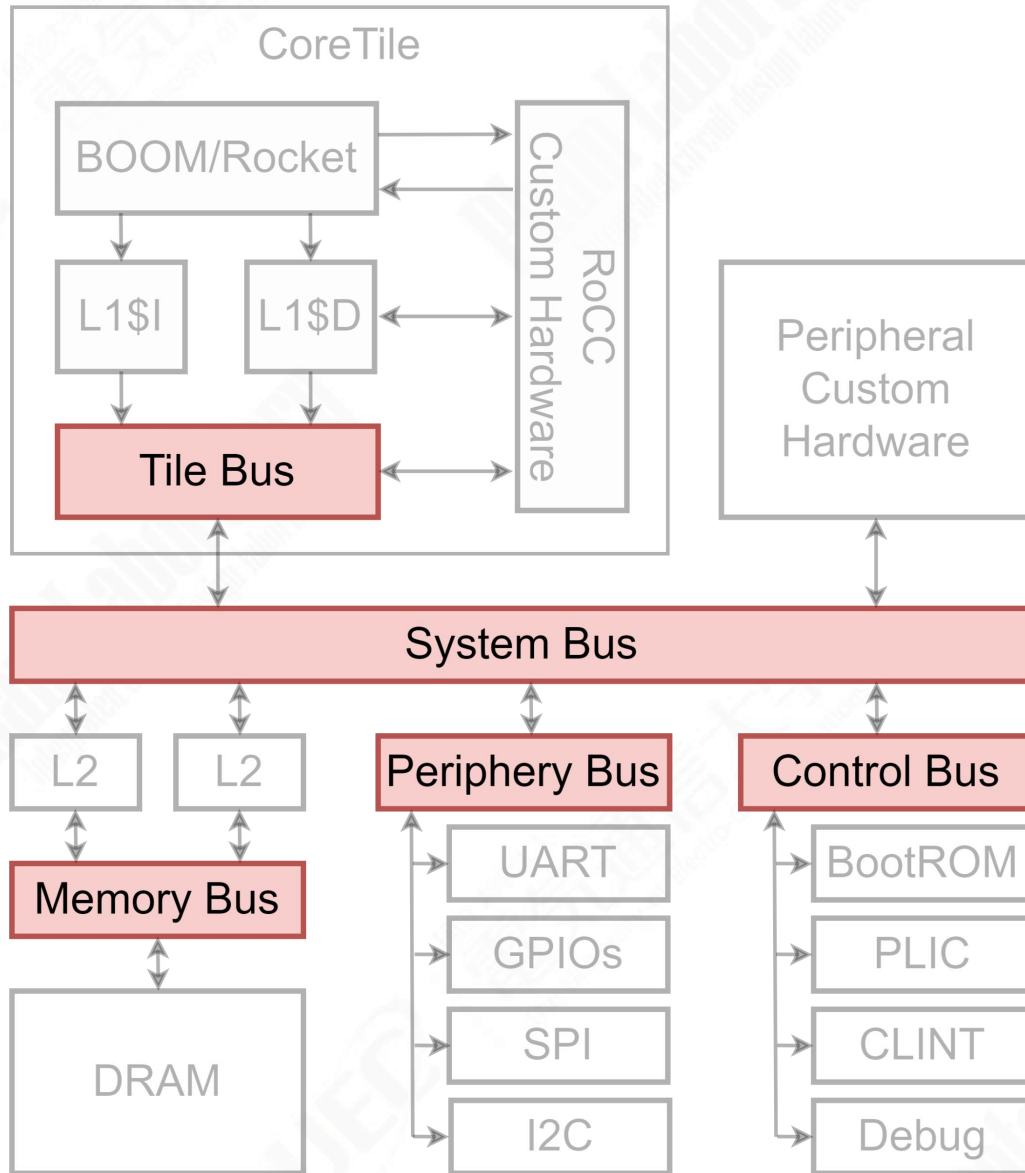
2. SoC Generation (3/11) SoC Architecture



Memory system:

- L1 Cache:
 - Separate Instruction Cache (\$I) and Data Cache (\$D).
 - Support different cache replacement policies.
- L2 Cache:
 - Tilelink-based.
 - Configurable capacity/banking.
- DRAM:
 - AXI-4 interface.
 - Support multiple banks.

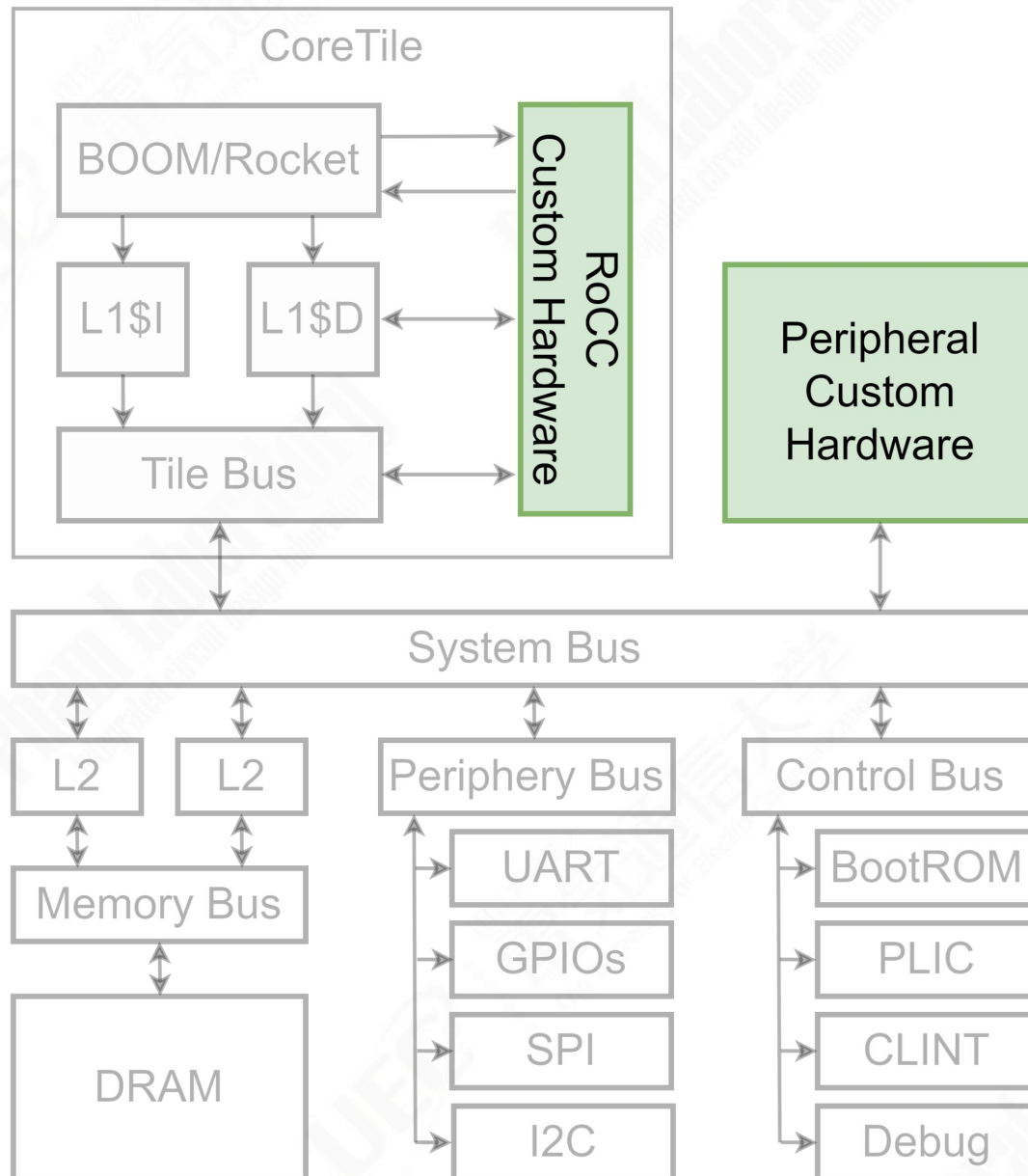
2. SoC Generation (4/11) SoC Architecture



Bus system:

- TileLink-based, an open-source chip-scale interconnect standard.
- Convertible to AXI/ACE.
- Support multi-core, accelerators, peripherals, DMA, atomic, etc.

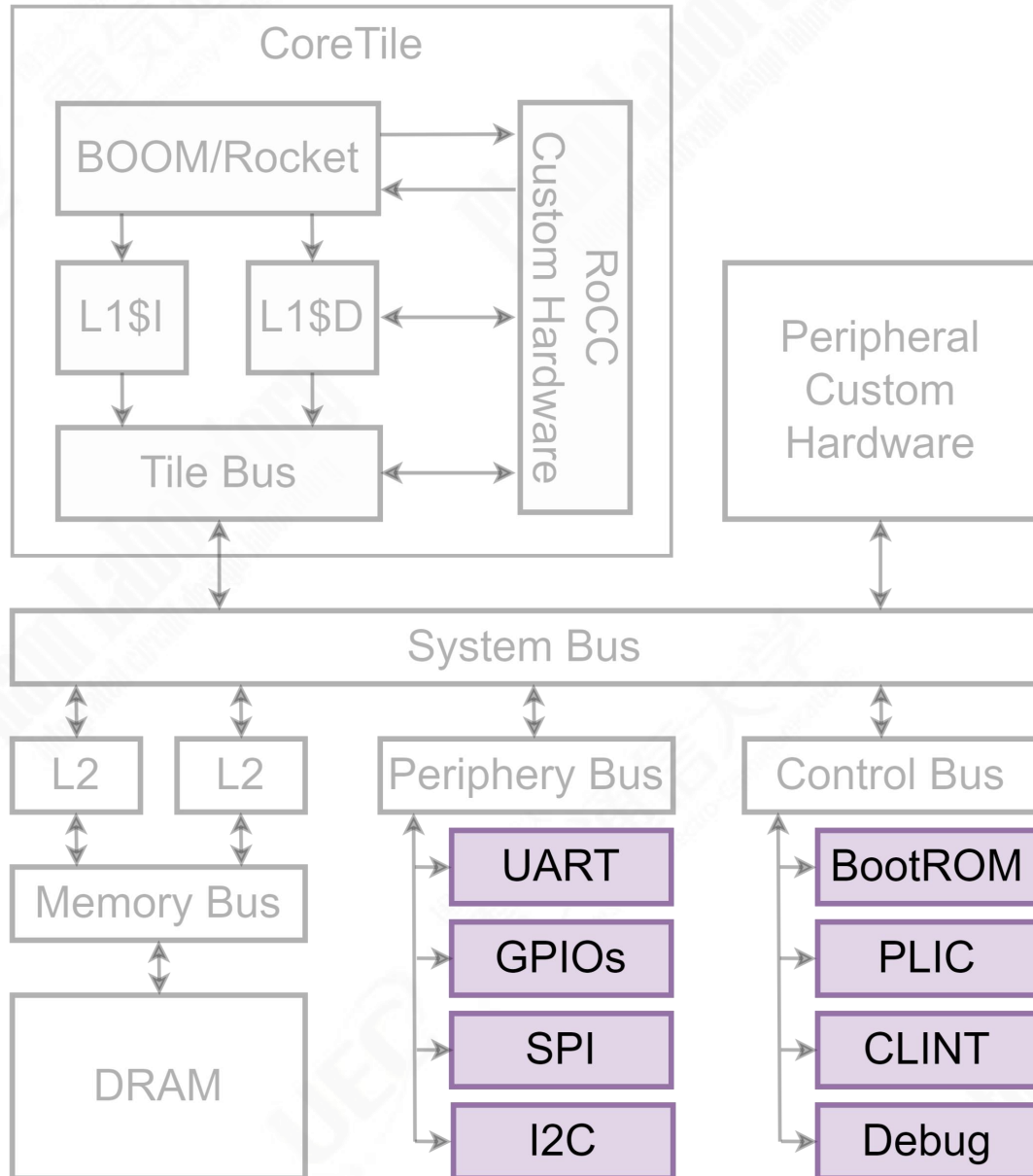
2. SoC Generation (5/11) SoC Architecture



Custom Hardware/Accelerator:

- Peripheral
 - Control by memory-mapped registers.
 - Supports DMA to memory system.
- RoCC:
 - Core automatically decode + sends custom instruction to accelerator.
 - Accelerator can support virtual-addressing.
 - Accelerator can write back into core registers.
 - Accelerator can fetch-from/write-to coherent L1 data cache or outer-memory.

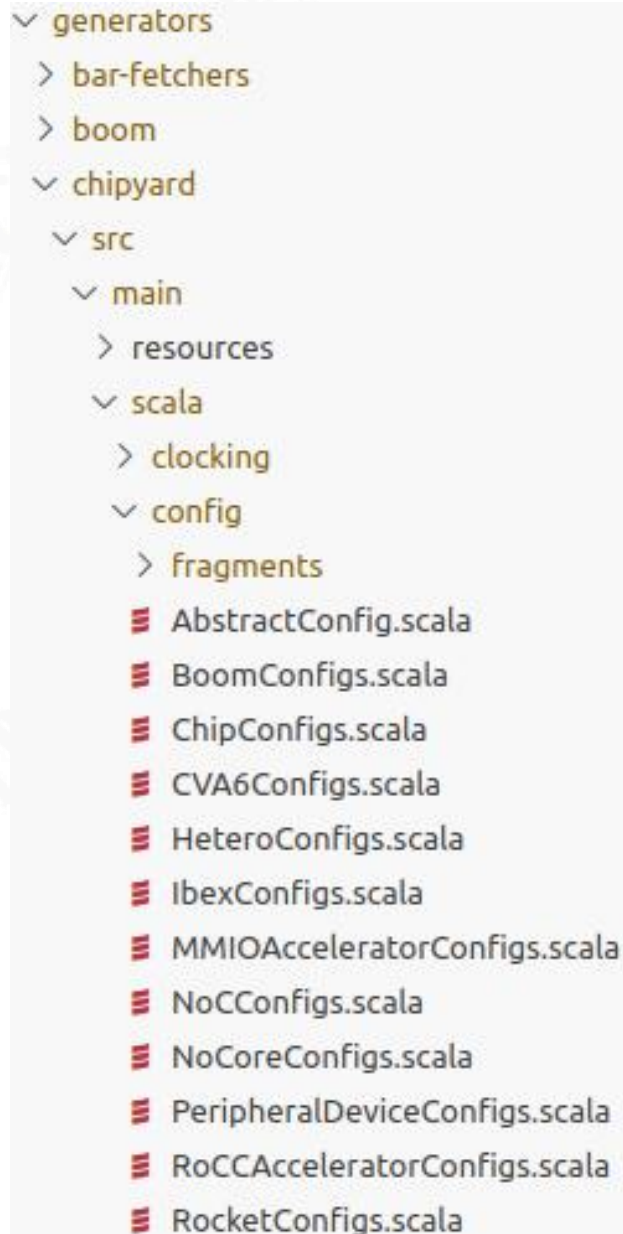
2. SoC Generation (6/11) SoC Architecture



Peripherals:

- Open-source common peripherals:
 - JTAG, Debugger, BootROM.
 - UARTs, GPIOs, SPI, I2C, PWM, etc.
- Support RISC-V interrupts:
 - Platform-Level Interrupt Controller (PLIC).
 - Core Local INTerrupt (CLINT).

2. SoC Generation (7/11) SoC Configuration



- The configurations of SoC provided by chipyard framework are stored in the path:

RISC-V-Demo/generators/chipyard/src/main/scala/config

- In this folder, you can see that there are many pre-defined configurations that you can use immediately.
- In case that you would like to customize your desired SoC, you can modify the corresponding config file in this folder. For example, in this case, I would like to build my SoC based on Rocket core. Therefore, I will add my new SoC configuration in file:

RISC-V-Demo/generators/chipyard/src/main/scala/config/RocketConfigs.scala

Note: you can modify any *xxxConfigs.scala* files except *AbstractConfig.scala*. This file define the abstract configuration for all other configurations of chipyard framework. Therefore modifying this file could cause unexpected faults.

2. SoC Generation (8/11) SoC Configuration

In this example, we will configure a SoC with:

- Tiny Rocket Core 32-bit.
- No-FPU

Step 1: Add a new class in file

RISC-V-Demo/generators/chipyard/src/main/scala/config/RocketConfigs.scala

```
class MyRocketConfig extends Config(  
  new chipyard.config.AbstractConfig  
)
```

Note: all configurations **must** extends from Config class and should begin with AbstractConfig.

2. SoC Generation (9/11) SoC Configuration

```
class With1TinyCore extends Config((site, here, up) => {
  case XLen => 32
  case TilesLocated(InSubsystem) => {
    val tiny = RocketTileParams(
      core = RocketCoreParams(
        useVM = false,
        fpu = None,
        mulDiv = Some(MulDivParams(mulUnroll = 8))),
    btb = None,
    dcache = Some(DCacheParams(
      rowBits = site(SystemBusKey).beatBits,
      nSets = 256, // 16Kb scratchpad
      nWays = 1,
      nTLBsets = 1,
      nTLBWays = 4,
      nMSHRs = 0,
      blockBytes = site(CacheBlockBytes),
      scratch = Some(0x80000000L))),
    icache = Some(ICacheParams(
      rowBits = site(SystemBusKey).beatBits,
```

Step 2: We need a tiny rocket core. You can found the definition for tiny rocket core in file:

RISC-V-Demo/generators/rocket-chip/src/main/scala/subsystem/Configs.scala

- In scala, when calling a class, the class' name should come after the name of the package. You can found the name of the package that contains class WithNSmallCores at the top of this file.
- Now we return to our SoC configuration to add the small rocket core.

```
class MyRocketConfig extends Config(
  new freechips.rocketchip.subsystem.With1TinyCore ++
  new chipyard.config.AbstractConfig
)
```


2. SoC Generation (10/11) SoC Configuration

Step 3: The default configuration of Small Rocket core is 64-bit, we need to override its definition to 32-bit. This class is *WithRV32* that locates at:

RISC-V-Demo/generators/rocket-chip/src/main/scala/subsystem/Configs.scala

```
class WithRV32 extends Config((site, here, up) => {  
  case XLen => 32  
  case TilesLocated(InSubsystem) => up(TilesLocated(InSubsystem), site) map {  
    case tp: RocketTileAttachParams => tp.copy(tileParams = tp.tileParams.copy(  
      core = tp.tileParams.core.copy(  
        fpu = tp.tileParams.core.fpu.map(_.copy(fLen = 32)),  
        mulDiv = Some(MulDivParams(mulUnroll = 8))))  
    case t => t  
  }  
})
```

To override the definition of class *WithNSmallCores*, class *WithRV32* **must** be put above class *WithNSmallCores* in our configuration *MyRocketConfig*.

```
class MyRocketConfig extends Config(  
  new freechips.rocketchip.subsystem.WithRV32 ++  
  new freechips.rocketchip.subsystem.With1TinyCore ++  
  new chipyard.config.AbstractConfig  
)
```

2. SoC Generation (11/11) SoC Configuration

Step 4: To remove FPU from the definition of Small Rocket Core, we need *WithoutFPU* in file: *RISC-V-Demo/generators/rocket-chip/src/main/scala/subsystem/Configs.scala*

```
class WithoutFPU extends Config((site, here, up) => {  
  case TilesLocated(InSubsystem) => up(TilesLocated(InSubsystem), site) map {  
    case tp: RocketTileAttachParams => tp.copy(tileParams = tp.tileParams.copy(  
      core = tp.tileParams.core.copy(fpu = None)))  
    case t => t  
  }  
})
```

To override the definition of class *WithNSmallCores*, class *WithoutFPU* **must** be put above class *WithNSmallCores* in our configuration *MyRocketConfig*.

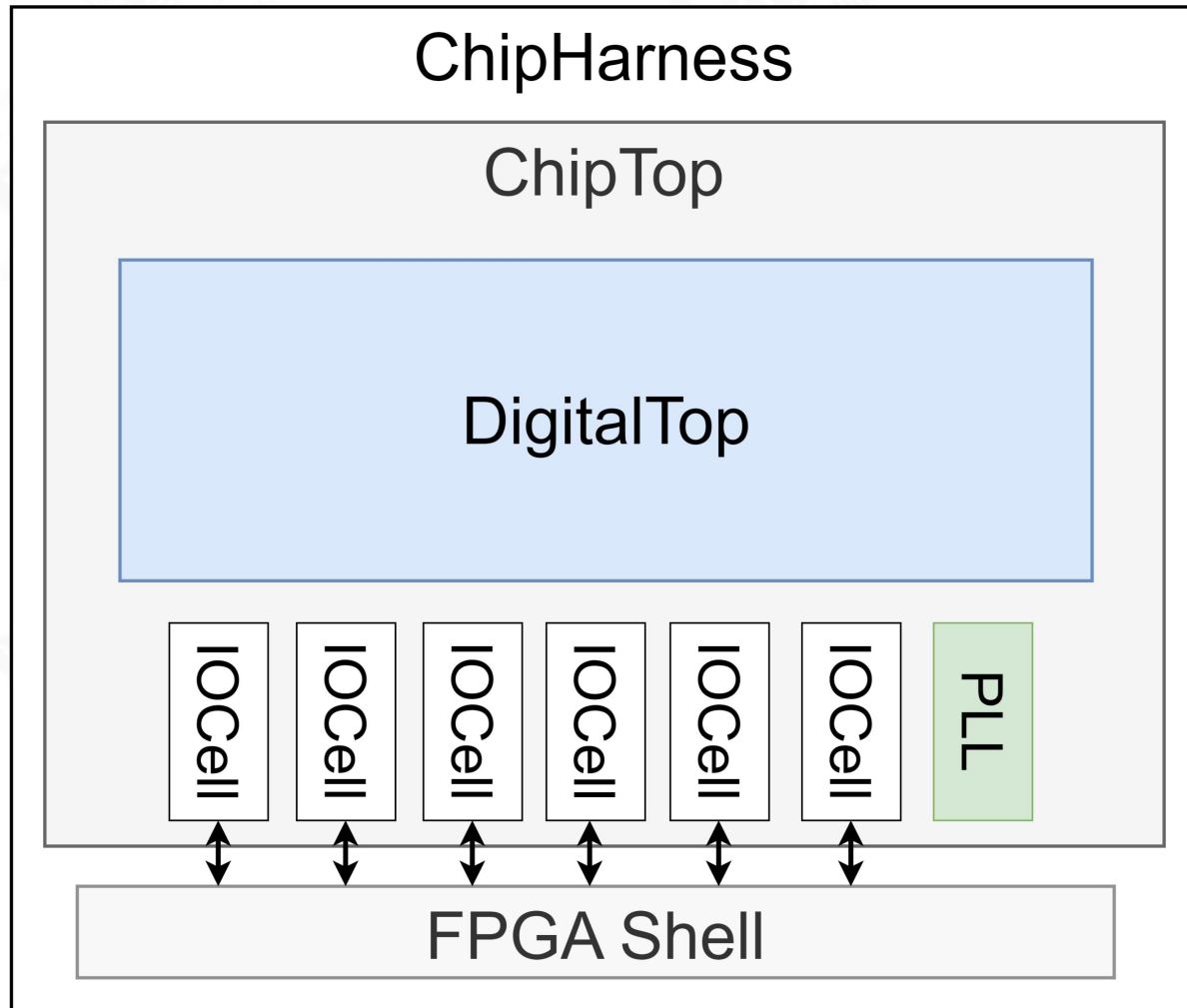
```
class MyRocketConfig extends Config(  
  new freechips.rocketchip.subsystem.WithoutFPU ++  
  new freechips.rocketchip.subsystem.WithRV32 ++  
  new freechips.rocketchip.subsystem.With1TinyCore ++  
  new chipyard.config.AbstractConfig  
)
```

At this step, we have already completed our desired SoC. This configuration will generate DigitalTop.

Outline

1. Framework organization
2. SoC Generation
3. **FPGA Prototyping**
4. Git clone and prepare
5. Make the system
6. Prepare SDCard
7. Using IDE
8. Modifying system by Scala config

3. FPGA Prototyping (1/10) Remind



To deploy our customized SoC in FPGA platform, we must follow the following step:

1. Integrate DigitalTop (which is our customized SoC).
2. Assign address map for peripherals in DigitalTop.
3. Connects DigitalTop to IOCell. This step is usually called IO Binders.
4. Connect the IOCell to FPGA Shell, which includes the IO of ChipHarness. Therefore, this step is usually called Harness Binders.
5. Define clock frequency for system.
6. Modify Makefile for our new configuration.

3. FPGA Prototyping (2/10) Step-by-step

The workspace for FPGA Prototyping is located at: *RISC-V-Demo/fpga*

In this example, we will deploy our SoC on Arty35T. Therefore, in *fpga* folder, we will create new configuration the location that store the Arty35T shell, which is: *RISC-V-Demo/fpga/src/main/scala/arty35T/Configs.scala*. The peripherals includes GPIOs, UART, SPI, and JTAG.

Step 1: Integrate DigitalTop (which is our customized SoC)

Firstly, we create a new configuration class that extends from Config class.

Our configuration class uses our recently created SoC configuration in the above section, which names *MyRocketConfig*.

```
class MyRocketArty32TConfig extends Config(  
  | new chipyard.MyRocketConfig  
)
```

Note: remember that the class name should follow the package name.

3. FPGA Prototyping (3/10) Step-by-step

Step 1*: (Additional step) After integrating *MyRocketConfig*, it is required to remove some high-resource consumption devices to fit with the limited resources of Arty35T board.

Please the four line from 77 to line 80 in order.

```
76 | class MyRocketArty32TConfig extends Config(  
77 |     new freechips.rocketchip.subsystem.WithIncoherentBusTopology ++ // use incoherent bus topology  
78 |     new chipyard.config.WithBroadcastManager ++ // remove L2$  
79 |     new freechips.rocketchip.subsystem.WithNBanks(0) ++ // remove L2$  
80 |     new freechips.rocketchip.subsystem.WithNoMemPort ++ // remove backing memory  
81 |     new chipyard.MyRocketConfig  
82 | )
```

Line 80 will remove memory port, which mean we will not have DDR and DDR controller.

Line 79 and 78 will remove L2 Cache it is useless after removing DDR.

Line 77 will remove the bus coherent manager. In our design, there is only one core that control the bus. Therefore, there is not inconsistency in the bus system. The bus cohenrence manager must be integrated when there are more than one master that control the bus (in case of multi-core or with additional accelerators).

3. FPGA Prototyping (4/10) Step-by-step

Step 2: Assign address map for peripherals in DigitalTop.

Take a look on provided class WithTinyArty35TTweaks from bottom up. It firstly call *WithSystemModifications* to remove unnecessary components in our example system.

```
class WithTinyArty35TTweaks extends Config(  
  // Other configurations  
  new WithDefaultPeripherals ++  
  new WithSystemModifications)
```



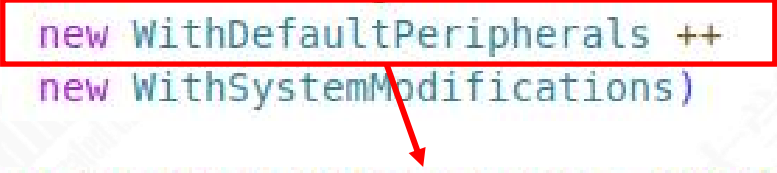
```
class WithSystemModifications extends Config((site, here, up) => {  
  case DTSTimebase => BigInt{(1e6).toLong}  
  case BootROMLocated(x) => up(BootROMLocated(x), site).map{ p =>  
    val freqMHz = (site(SystemBusKey).dtsFrequency.get / (1000 * 1000)).toLong  
    // Make sure that the bootrom is always rebuilt  
    val clean = s"make -C fpga/src/main/resources/arty35t/sdboot clean"  
    require (clean.!! == 0, "Failed to clean")  
    // Build the bootromt  
    val make = s"make -C fpga/src/main/resources/arty35t/sdboot XLEN=${site(XLen)} PBUS_CLK=${freqMHz}"  
    require (make.!! == 0, "Failed to build bootrom")  
    p.copy(hang = 0x10000, contentFileName = s"./fpga/src/main/resources/arty35t/sdboot/build/sdboot.bin")  
  }  
}
```

3. FPGA Prototyping (5/10) Step-by-step

Step 2: Assign address map for peripherals in DigitalTop.

Then it call *WithDefaultPeripherals* to modify address map of integrated peripherals. You can customized the address depend on your SoC through this class. In this case, only UART, SPI, GPIO have addresses. JTAG is the master of the bus system. Therefore, it does not have addresses.

```
class WithTinyArty35TTweaks extends Config(  
  // Other configurations  
  new WithDefaultPeripherals ++  
  new WithSystemModifications)
```



```
class WithDefaultPeripherals extends Config((site, here, up) => {  
  case PeripheryUARTKey => List(UARTParams(address = BigInt(0x64000000L)))  
  case PeripherySPIKey => List(SPIParams(rAddress = BigInt(0x64001000L)))  
  case PeripheryGPIOKey => List(GPIOParams(address = BigInt(0x64002000L), width = 24))  
})
```


3. FPGA Prototyping (6/10) Step-by-step

Step 3: Connects DigitalTop to IOCell (IO Binders).

The IOBinder classes are defined in

RISC-V-Demo/fpga/src/main/scala/arty35t/IOBinders.scala

Depend on the kind of peripherals that are integrated in your customized SoC, you add the corresponding IOBinder class. In this example, we add *WithTLIOPassthrough* for the peripherals that have master port on bus system (JTAG), *WithSPIIOPassthrough* for SPI, *WithUARTIOPassthrough* for UART, and *WithGPIOIOPassthrough* for GPIO.

```
class WithTinyArty35TTweaks extends Config(  
  60 // IO Binders  
  61 new WithGPIOIOPassthrough ++  
  62 new WithUARTIOPassthrough ++  
  63 new WithSPIIOPassthrough ++  
  64 new WithTLIOPassthrough ++
```

3. FPGA Prototyping (7/10) Step-by-step

Step 4: Connect the IOCell to FPGA Shell (Harness Binders).

The HarnessBinder classes are defined in

RISC-V-Demo/fpga/src/main/scala/arty35t/HarnessBinders.scala

Depend on the kind of peripherals that are integrated in your customized SoC, you add the corresponding HarnessBinder class. In this example, we add

WithArty35TJTAGHarnessBinder for the peripherals that have master port on bus system (JTAG), *WithArty35TSPISDCardHarnessBinder* for SPI, *WithArty35TUARTHarnessBinder* for UART, and *WithArty35TGPIOHarnessBinder* for GPIO.

```
class WithTinyArty35TTweaks extends Config(  
  // Harness Binder  
  new WithArty35TUARTHarnessBinder ++  
  new WithArty35TSPISDCardHarnessBinder ++  
  new WithArty35TJTAGHarnessBinder ++  
  new WithArty35TGPIOHarnessBinder ++
```


3. FPGA Prototyping (8/10) Step-by-step

Step 5: Define clock frequency for system.

At this step, we define for:

- Input clock frequency: *WithHarnessBinderClockFreqMHz*
- Expected Memory bus frequency: *WithMemoryBusFrequency*
- Expected System bus frequency: *WithSystemBusFrequency*
- Expected PeripheryBusFrequency: *WithPeripheryBusFrequency*

```
class WithTinyArty35TTweaks extends Config(  
  // Clock configs  
  new chipyard.harness.WithAllClocksFromHarnessClockInstantiator ++  
  new chipyard.harness.WithHarnessBinderClockFreqMHz(50) ++  
  new chipyard.config.WithMemoryBusFrequency(50.0) ++  
  new chipyard.config.WithSystemBusFrequency(50.0) ++  
  new chipyard.config.WithPeripheryBusFrequency(50.0) ++  
  new chipyard.harness.WithAllClocksFromHarnessClockInstantiator ++  
  new chipyard.clocking.WithPassthroughClockGenerator ++
```

3. FPGA Prototyping (9/10) Step-by-step

Step 5: Define clock frequency for system.

After that, we add the *WithTinyArty35TTweaks* into our configuration

```
class MyRocketArty32TConfig extends Config(  
  new WithTinyArty35TTweaks ++  
  new freechips.rocketchip.subsystem.WithIncoherentBusTopology ++  
  new chipyard.config.WithBroadcastManager ++  
  new freechips.rocketchip.subsystem.WithNBanks(0) ++  
  new freechips.rocketchip.subsystem.WithNoMemPort ++  
  new chipyard.MyRocketConfig  
)
```

3. FPGA Prototyping (10/10) Step-by-step

Step 6: Finally, we add new entry into Makefile on:

RISC-V-Demo/fpga/Makefile

```
ifeq ($(SUB_PROJECT) myarty)
# TODO: Fix with Arty
SBT_PROJECT      ?= fpga_platforms
MODEL            ?= Arty35TwoDDRHarness
VLOG_MODEL       ?= Arty35TwoDDRHarness
MODEL_PACKAGE    ?= chipyard.fpga.arty35t
CONFIG           ?= MyRocketArty32TConfig
CONFIG_PACKAGE   ?= chipyard.fpga.arty35t
GENERATOR_PACKAGE ?= chipyard
TB               ?= none # unused
TOP              ?= ChipTop
BOARD            ?= arty
FPGA_BRAND       ?= xilinx
endif
```

We only need to change 2 fields:

Blue field: any name without overlap with other entries.

Red field: name of the configuration class that you have just created. In this example, it is *MyRocketArty32TConfig*.

You can now generate the corresponding bitstream by \$ make bitstream At *RISC-V-Demo/fpga/*

```
$ make SUB_PROJECT=MyRocketArty32TConfig bitstream
```

Outline

1. Framework organization
2. SoC Generation
3. FPGA Prototyping
4. **Git clone and prepare**
5. Make the system
6. Prepare SDCard
7. Using IDE
8. Modifying system by Scala config

4. Git clone and prepare (1/3) Build the framework

- Download the installer using [curl](#) or [wget](#) or your favorite program and run the script.

Using [curl](#)

```
$ curl -L -O "https://github.com/conda-  
forge/miniforge/releases/latest/download/Miniforge3-$(uname -  
m).sh"  
$ bash Miniforge3-$(uname -m).sh
```

Or using [wget](#)

```
$ wget "https://github.com/conda-  
forge/miniforge/releases/latest/download/Miniforge3-$(uname -  
m).sh"  
$ bash Miniforge3-$(uname -m).sh
```

Then install [conda-lock](#)

```
$ conda create -n "myenv" python=3.9.0  
$ conda install -n myenv conda-lock==1.4.0
```

4. Git clone and prepare (2/3) Build the framework

- To clone the project:

```
$ https://github.com/binhkieudo/RISC-V-Demo.git  
$ cd RISC-V-Demo
```

- Conda environment must be activated before building the platform:

```
$ conda activate myenv
```

- Then start building the framework:

```
$ echo y | ./build-setup.sh -s 8 -s 9 riscv-tools
```

Note 1: You only need to build the framework for the first use.

Note 2: Internet is required to build the framework.

Note: the conda environment must be activated at the root folder of RISC-V-Demo

4. Git clone and prepare (3/3) Prepare Arty-A7 license

- Arty-A7 license is free. You can download and install the Arty-A7 license to Vivado.
 - Download the link from the Digilent [website](https://www.digilent.com).
 - Guide to install the license:

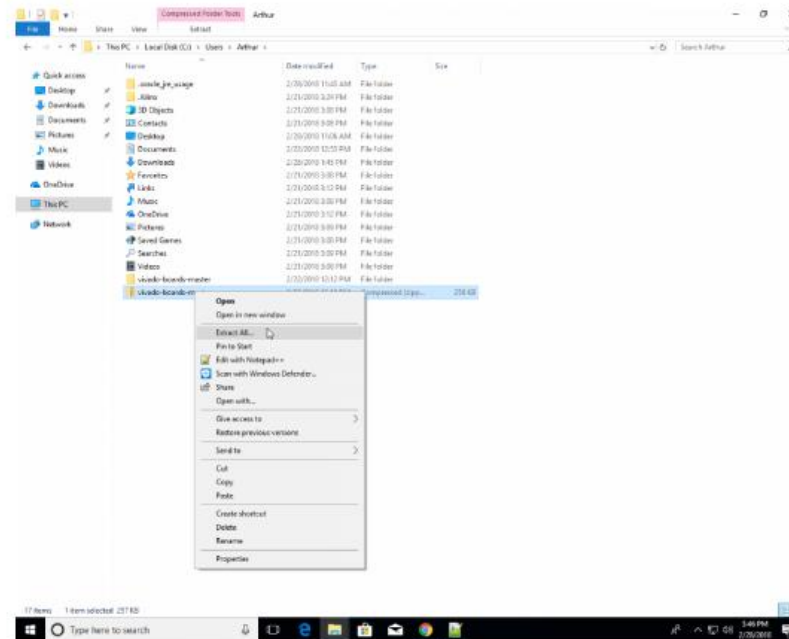
<https://digilent.com/reference/programmable-logic/guides/installing-vivado-and-vitis>

Install Digilent's Board Files

Digilent provides *board files* for each FPGA development board. These files make it easy to select the correct part when creating a new project and allow for automated configuration of several complicated components (including the Zynq Processing System and Memory Interface Generator) used in many designs.

The board files will be copied into your version of Vivado's installation directory. At the end of this section, an alternate method of installation is presented, which users familiar with git may find more convenient.

Download the most recent **Master Branch ZIP Archive** of Digilent's [vivado-boards](https://github.com/digilent/vivado-boards) Github repository and extract it.

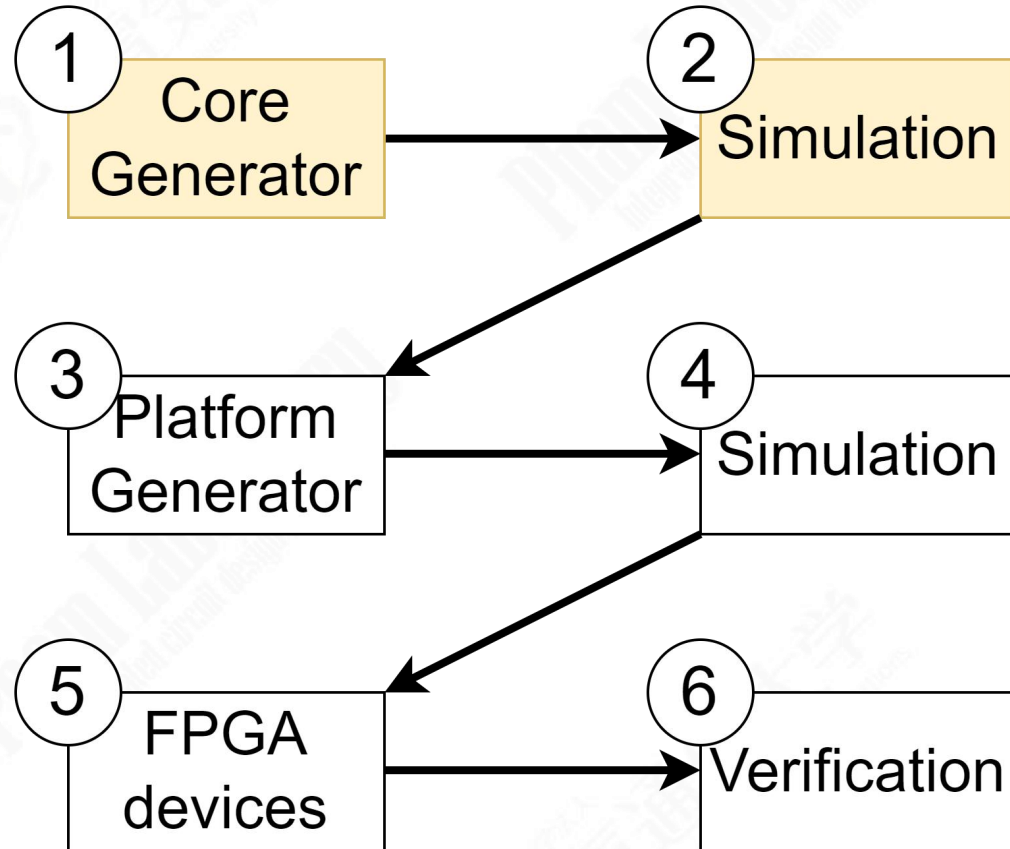


1. Extract the ZIP file
2. Copy all the content in new/board_files
3. Paste them to Xilinx/Vivado/2022.1/data/boards/board_files/

Outline

1. Framework organization
2. SoC Generation
3. FPGA Prototyping
4. Git clone and prepare
5. **Make the system**
6. Prepare SDCard
7. Using IDE
8. Modifying system by Scala config

5. Make the system (1/15) Platform structure



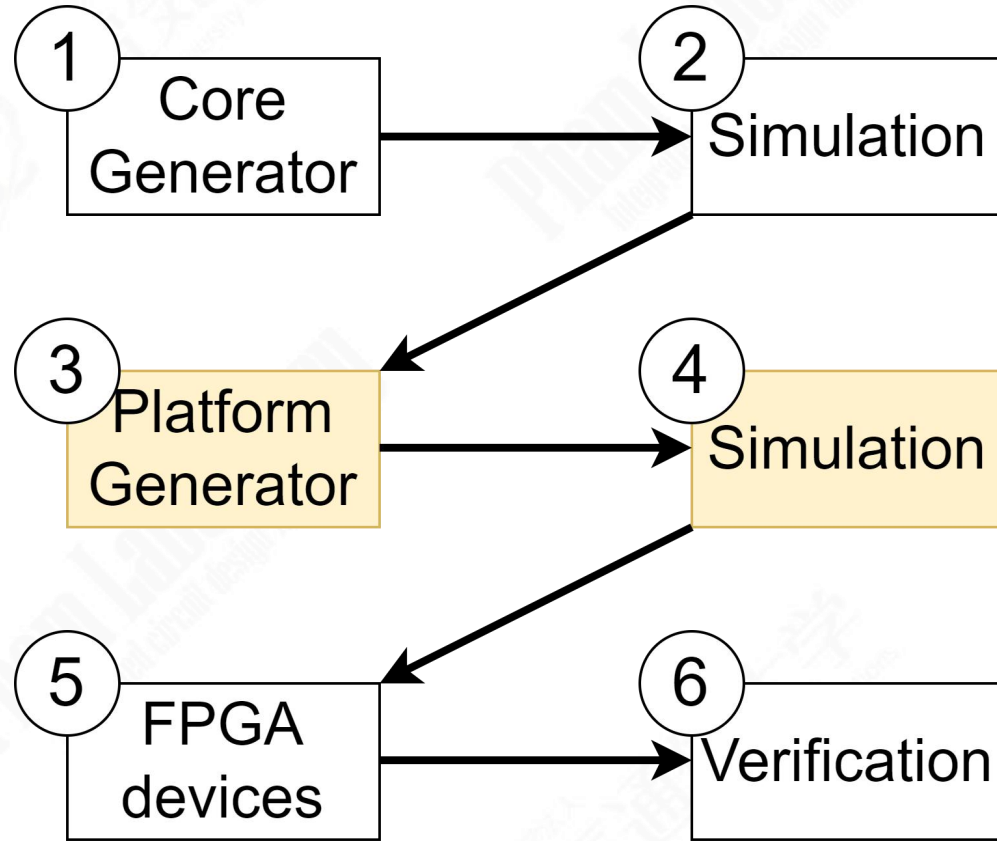
1. Core Generator (Abstract):

- This step focuses on the design of the internal RISC-V system (RISC-V cores' type, number of cores, bus architecture, etc).
- Fake peripherals (GPIOs, SPI, memory, etc) are integrated in this step.

2. Simulation (without peripherals)

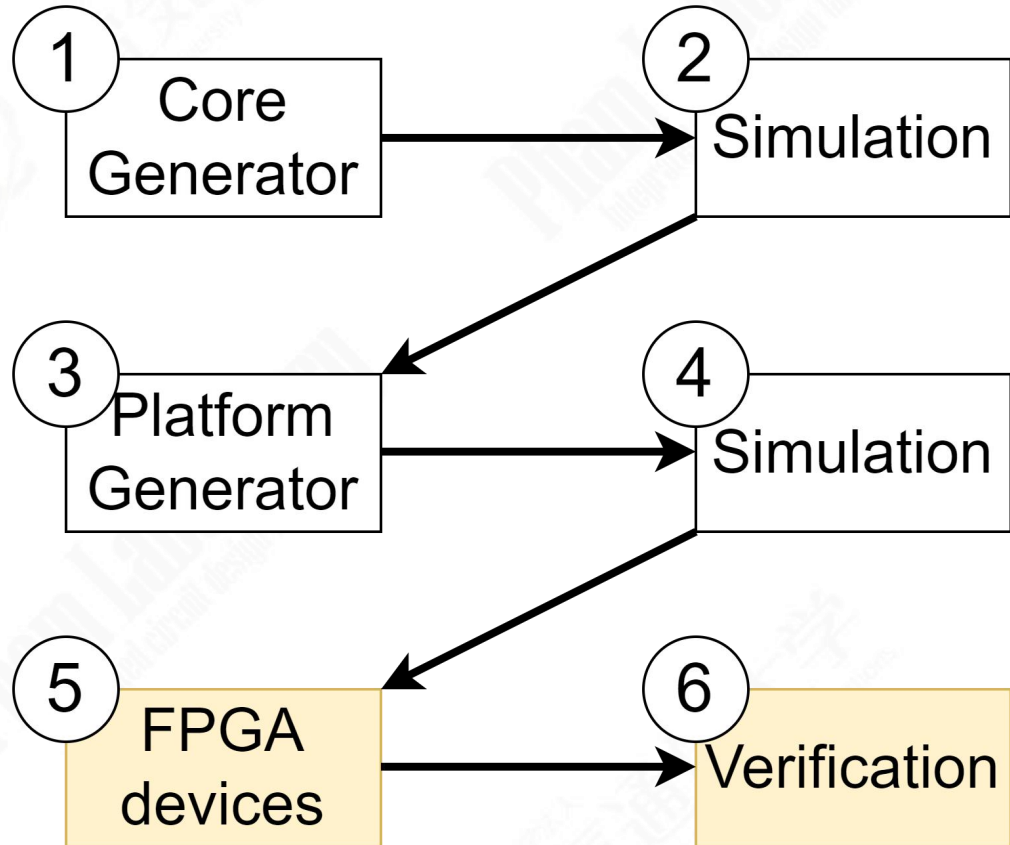
- Verilator is mainly used to simulate the internal system behavior and generate the corresponding waveform.

5. Make the system (2/15) Platform structure



3. Platform Generator (Physical deployment):
 - This step focuses on integrating the generated internal system on a specific platform.
 - Platform specifications are configured separately with core design.
 - Designers will map the generated internal system with physical platform design.
4. Simulation (with peripherals)
 - Verilator, VSIM, VCS, Xcelium.
 - Fake peripherals are removed in this step. Designers need to provide their testbench.
 - The simulator must be connected to the cell library of the deployed platform.

5. Make the system (3/15) Platform structure



5. FPGA devices:

- Bitstream is programmed into FPGA devices.

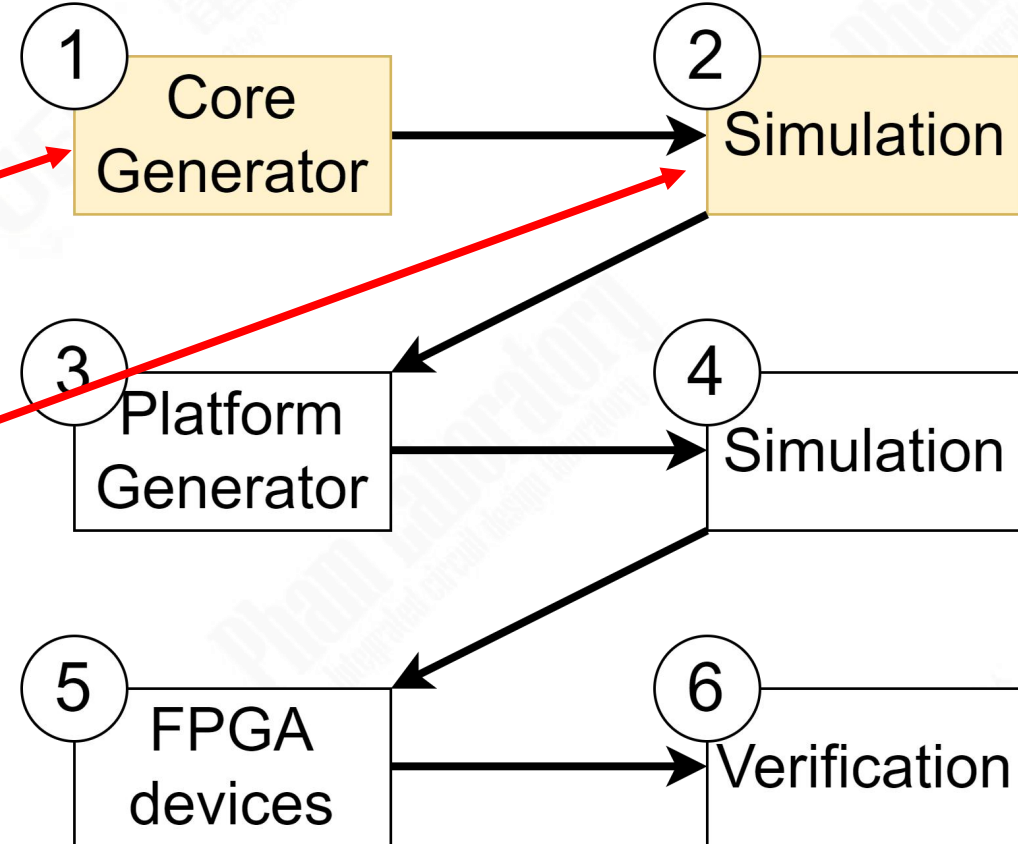
6. Verification

- Integrated Logic Analyzer (ILA) on FPGA device.
- Debugger (through OpenOCD and gdb).
- Benchmark.

5. Make the system (4/15) Framework architecture



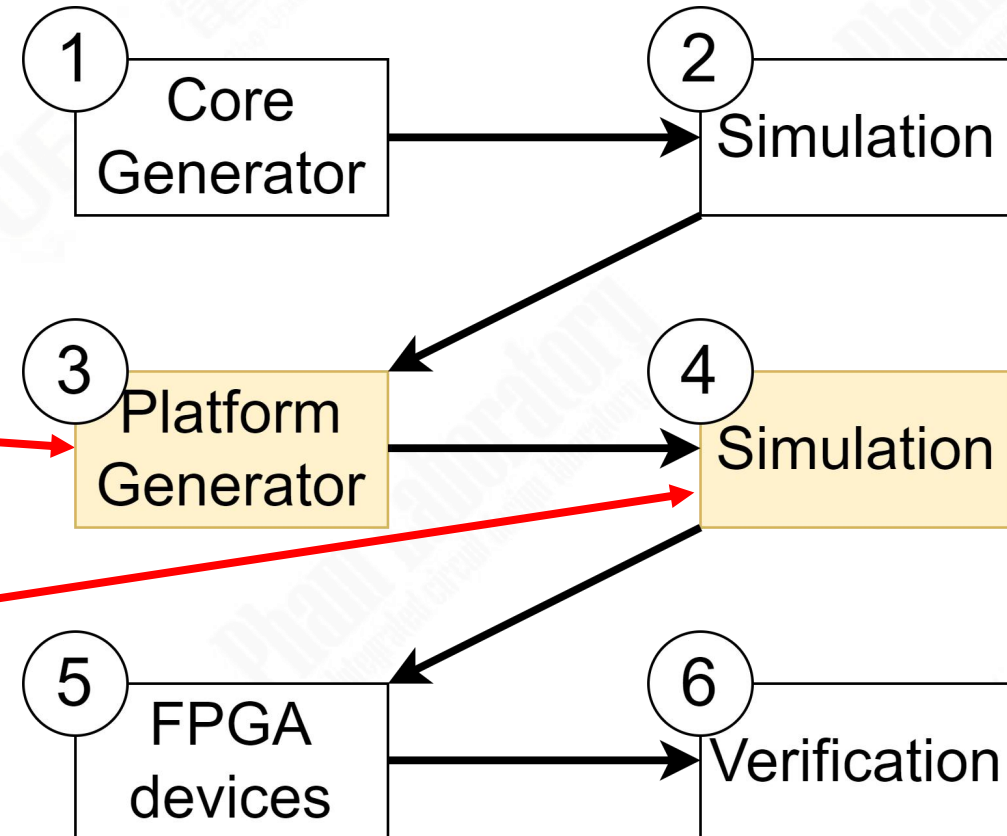
- build-libfesvr.a.log
- build.sbt
- build-setup.log
- build-setup.sh -> scripts/build-setup.sh
- CHANGELOG.md
- common.mk
- conda-reqs
- CONTRIBUTING.md
- dockerfiles
- docs
- env.sh
- fpga
- **generators**
- init-submodules-no-riscv-tools.log
- LICENSE
- LICENSE.SiFive
- project
- README.md
- scripts
- **sims**
- software
- tags
- target
- tests
- toolchains
- tools
- variables.mk
- vlsi



5. Make the system (5/15) Framework architecture



- build-libfesvr.a.log
- build.sbt
- build-setup.log
- build-setup.sh -> scripts/build-setup.sh
- CHANGELOG.md
- common.mk
- conda-reqs
- CONTRIBUTING.md
- dockerfiles
- docs
- env.sh
- **fpga**
- generators
- init-submodules-no-riscv-tools.log
- LICENSE
- LICENSE.SiFive
- project
- README.md
- scripts
- **sims**
- **software**
- tags
- target
- tests
- toolchains
- tools
- variables.mk
- vlsi



5. Make the system (6/15) Framework architecture

Makefile

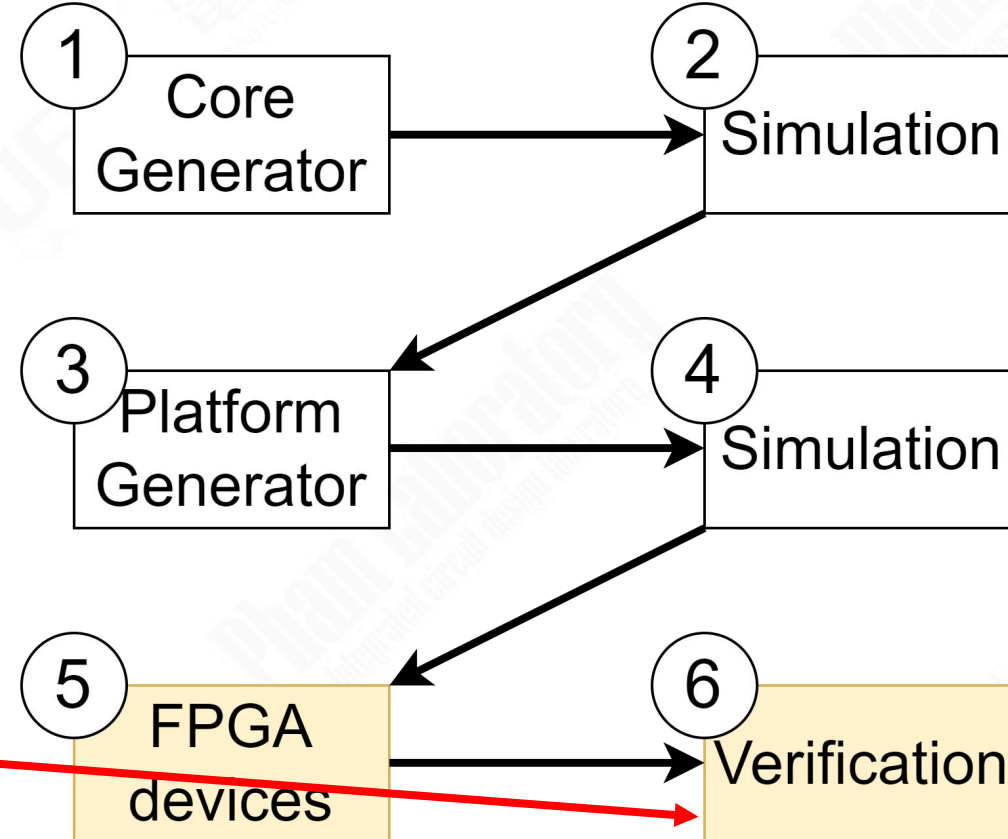
SBT (.scala)

FIRRTL (.fir)

FPGA (.v)

FPGA (.bit)

```
build-libfesvr.a.log
build.sbt
build-setup.log
build-setup.sh -> scripts/build-setup.sh
CHANGELOG.md
common.mk
conda-reqs
CONTRIBUTING.md
dockerfiles
docs
env.sh
fpga
generators
init-submodules-no-riscv-tools.log
LICENSE
LICENSE.SiFive
project
README.md
scripts
sims
software
tags
target
tests
toolchains
tools
variables.mk
vlsi
```



5. Make the system (7/15) Manager with SBT

Makefile

SBT (*.scala*)

FIRRTL (*.fir*)

FPGA (*.v*)

FPGA (*.bit*)

At RISC-V-Demo/build.sbt:

```
lazy val fpga_platforms = (project in file "./fpga")  
  .dependsOn(chipyard, fpga_shells)  
  .settings(commonSettings)
```

Many “Physical Platforms” could be developed separately with the integrated cores.

```
class WithArty100TTweaks extends Config(  
  new chipyard.harness.WithAllClocksFromHarnessClockInstantiator ++  
  new WithArty100UARTTSI ++  
  new WithArty100TDDRRTL ++  
  new WithNoDesignKey ++  
  new chipyard.harness.WithHarnessBinderClockFreqMHz(50) ++  
  new chipyard.config.WithMemoryBusFrequency(50.0) ++  
  new chipyard.config.WithSystemBusFrequency(50.0) ++  
  new chipyard.config.WithPeripheryBusFrequency(50.0) ++  
  new chipyard.harness.WithAllClocksFromHarnessClockInstantiator ++  
  new chipyard.clocking.WithPassthroughClockGenerator ++  
  new chipyard.config.WithNoDebug ++ // no jtag  
  new chipyard.config.WithNoUART ++ // use UART for the UART-TSI thing instad  
  new chipyard.config.WithTLBackingMemory ++ // FPGA-shells converts the AXI to TL for us  
  new freechips.rocketchip.subsystem.WithExtMemSize(BigInt(256) << 20) ++ // 256mb on ARTY  
  new freechips.rocketchip.subsystem.WithoutTLMonitors)
```

RISC-V-Demo/generators/chipyard/src/main/scala/config/RocketConfigs.scala

5. Make the system (8/15) Manager with SBT

Makefile

SBT (.scala)

FIRRTL (.fir)

FPGA (.v)

FPGA (.bit)

At RISC-V-Demo/build.sbt:

```
lazy val chipyard = (project in file("generators/chipyard"))
  .dependsOn(testchipip, rocketchip, boom, hwacha, sifive_blocks, sifive_cache, iocell,
    sha3, // On separate line to allow for cleaner tutorial-setup patches
    dsptools, `rocket-dsp-utils`,
    gemmini, icenet, tracegen, cva6, nvdla, sodor, ibex, fft_generator,
    constellation, mempress, barfi, shuttle)
  .settings(libraryDependencies += rocketLibDeps.value)
  .settings(
    libraryDependencies += Seq(
      "org.reflections" % "reflections" % "0.10.2"
    )
  )
  .settings(commonSettings)
```

Many “Abstract Cores” could be developed separately with the deployed platform.

- Many designs can be deployed in a platform.
- Many platforms can be used for a design.

RISC-V-Demo/generators/chipyard/src/main/scala/config/RocketConfigs.scala

```
class RocketConfig extends Config(
  new freechips.rocketchip.subsystem.WithNBigCores(1) ++ // single rocket-core
  new chipyard.config.AbstractConfig)
```


5. Make the system (9/15) Scala structure



FPGA Shell folder	FPGA folder	
GPIO Pins UART Pins SPI Pins I2C Pins DDR Ports SDRAM Ports CODEC Ports JTAG Pins Other Ports	RVC. System	RVC. Subsystem
	● General Purpose IO ● UART ● SPI Flash ● I2C ● SDRAM/DDR ● TL Serial (For simulations) ● FFT/CODEC ● Any additional peripherals	● TileLink Buses ● Debug Module ● Boot ROM ● Core Local Interrupts ● Platform Level Interrupt Controller ● Coreplex ○ Rocket ○ BOOM

5. Make the system (10/15) Make bitstream



Now, to make the system, go to the build folder:

```
$ cd fpga
```

Export Vivado to the **PATH**:

```
$ export PATH=/opt/Xilinx/Vivado/<version>/bin/:$PATH
```

For the compilation:

```
$ make SUB_PROJECT=myarty verilog
```

This will compile Scala to Verilog (*also compile the boot ROM C/C++ sources*)

Or in case that you would like to generate bitstream:

```
$ make SUB_PROJECT=myarty bitstream
```

This will compile Scala to Verilog then bitstream file.

5. Make the system (11/15) Make bitstream



After generating bitstream, the **generated-src** folder is created:

Inside the **generated-src** folder, there are many files:
Verilog files, FIRRTL files, temporary Java files, boot ROM files, device tree, etc.

```
Arty35TwoDDRHarness.anno.json
Arty35TwoDDRHarness.cache
Arty35TwoDDRHarness.hw
Arty35TwoDDRHarness.in_user_files
Arty35TwoDDRHarness.xpr
bootrom.rv32.img
bootrom.rv64.img
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x0.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x0.1.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x100000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x110000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x2000000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x4000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x40.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x64000000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x64001000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x64002000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0xc000000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.all.f
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.anno.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.appended.anno.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.bb.f
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.chisel.log
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.d
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.dromajo_params.h
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.dts
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.extra.anno.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.extra.sfc.anno.json
```

```
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.extrasfc.anno.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.fir
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.graphml
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.harnessSysPLL.vivado.tcl
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.memmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.mems.conf
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.model.f
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.model.mems.conf
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.plusArgs
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.rom.conf
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.sfc.anno.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.sfc.fir
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.shell.sdc
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.shell.vivado.tcl
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.shell.xdc
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.top.f
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.top.mems.conf
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.vsrcs.f
clockInfo.txt
gen-collateral
model_module_hierarchy.json
model_module_hierarchy.uniquified.json
obj
sim_files.common.f
sim_files.f
top_module_hierarchy.json
vivado.log
```


5. Make the system (12/15) Make bitstream

- .xpr: you can open the Vivado project directly from this file.
- .dts: device tree
- .gen-collateral: contains all Verilog/system Verilog sources.
- .obj: contains bitstream and reports (resources usage, timing, etc).

```
Arty35TwoDDRHarness.anno.json
Arty35TwoDDRHarness.cache
Arty35TwoDDRHarness.hw
Arty35TwoDDRHarness.in_user_files
Arty35TwoDDRHarness.xpr
bootrom.rv32.img
bootrom.rv64.img
```

```
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x0.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x0.1.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x100000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x110000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x2000000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x4000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x40.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x64000000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x64001000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0x64002000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.0xc000000.0.regmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.all.f
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.anno.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.appended.anno.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.bb.f
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.chisel.log
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.d
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.dromajo_params.h
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.dts
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.extra.anno.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.extra.sfc.anno.json
```

```
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.extrasfc.anno.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.fir
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.graphml
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.harnessSysPLL.vivado.tcl
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.memmap.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.mems.conf
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.model.f
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.model.mems.conf
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.plusArgs
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.rom.conf
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.sfc.anno.json
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.sfc.fir
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.shell.sdc
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.shell.vivado.tcl
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.shell.xdc
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.top.f
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.top.mems.conf
chipyard.fpga.arty35t.Arty35TwoDDRHarness.SmallRocketArty35TConfig.vsrcs.f
clockInfo.txt
gen-collateral
model_module_hierarchy.json
model_module_hierarchy.uniquified.json
obj
sim_files.common.f
sim_files.f
top_module_hierarchy.json
vivado_log
```

5. Make the system (13/15) Verilog files

Makefile

SBT (.scala)

FIRRTL (.fir)

FPGA (.v)

FPGA (.bit)

```
module Arty35TwoDDRHarness(  
  input sys_clock,  
  inout jtag_jtag_TCK,  
  jtag_jtag_TMS,  
  jtag_jtag_TDI,  
  jtag_jtag_TDO,  
  jtag_srst_n,  
  uart_txd,  
  uart_rxd,  
  gpio_gpio_0,  
  gpio_gpio_1,  
  gpio_gpio_2,  
  gpio_gpio_3,  
  gpio_gpio_4,  
  gpio_gpio_5,  
  gpio_gpio_6,  
  gpio_gpio_7,  
  gpio_gpio_8,  
  gpio_gpio_9,  
  gpio_gpio_10,  
  gpio_gpio_11,  
  gpio_gpio_12,  
  gpio_gpio_13,  
  gpio_gpio_14,  
  gpio_gpio_15,  
  gpio_gpio_16,  
  gpio_gpio_17,  
  gpio_gpio_18,  
  gpio_gpio_19,  
  gpio_gpio_20,  
  gpio_gpio_21,  
  gpio_gpio_22,  
  gpio_gpio_23,  
  sdio_spi_clk,  
  sdio_spi_cs,  
  sdio_spi_dat_0,  
  sdio_spi_dat_1,  
  sdio_spi_dat_2,  
  sdio_spi_dat_3,  
  input reset  
);
```

Top file:

Arty35TwoDDRHarness.sv

Note: All the source files
are inside **gen-collateral**
folder

File that contains all
the memories used in
the system:

chipyard.fpga.art
y35t.Arty35TwoDDR
Harness.SmallRock
etArty35TConfig.t
op.mems.v

```
module data_arrays_0_ext(  
  input [11:0] RW0_addr,  
  input RW0_clk,  
  input [31:0] RW0_wdata,  
  output [31:0] RW0_rdata,  
  input RW0_en,  
  input RW0_wmode,  
  input [3:0] RW0_wmask  
);  
wire [11:0] mem_0_0_RW0_addr;  
wire mem_0_0_RW0_clk;  
wire [7:0] mem_0_0_RW0_wdata;  
wire [7:0] mem_0_0_RW0_rdata;  
wire mem_0_0_RW0_en;  
wire mem_0_0_RW0_wmode;  
wire mem_0_0_RW0_wmask;  
wire [11:0] mem_0_1_RW0_addr;  
wire mem_0_1_RW0_clk;  
wire [7:0] mem_0_1_RW0_wdata;  
wire [7:0] mem_0_1_RW0_rdata;  
wire mem_0_1_RW0_en;  
wire mem_0_1_RW0_wmode;  
wire mem_0_1_RW0_wmask;  
wire [11:0] mem_0_2_RW0_addr;  
wire mem_0_2_RW0_clk;  
wire [7:0] mem_0_2_RW0_wdata;  
wire [7:0] mem_0_2_RW0_rdata;  
wire mem_0_2_RW0_en;  
wire mem_0_2_RW0_wmode;  
wire mem_0_2_RW0_wmask;  
wire [11:0] mem_0_3_RW0_addr;  
wire mem_0_3_RW0_clk;  
wire [7:0] mem_0_3_RW0_wdata;  
wire [7:0] mem_0_3_RW0_rdata;  
wire mem_0_3_RW0_en;  
wire mem_0_3_RW0_wmode;  
wire mem_0_3_RW0_wmask;  
wire [7:0] RW0_rdata_0_0 = mem_0_0_RW0_rdata;  
wire [7:0] RW0_rdata_0_1 = mem_0_1_RW0_rdata;  
wire [7:0] RW0_rdata_0_2 = mem_0_2_RW0_rdata;  
wire [7:0] RW0_rdata_0_3 = mem_0_3_RW0_rdata;  
wire [15:0] _GEN_0 = {RW0_rdata_0_1,RW0_rdata_0_0  
wire [23:0] _GEN_1 = {RW0_rdata_0_2,RW0_rdata_0_1  
wire [31:0] RW0_rdata_0 = {RW0_rdata_0_3,RW0_rdat  
wire [15:0] _GEN_2 = {RW0_rdata_0_1,RW0_rdata_0_0  
wire [23:0] _GEN_3 = {RW0_rdata_0_2,RW0_rdata_0_1  
endmodule data_arrays_0_ext mem_0_0_0;
```


5. Make the system (14/15) Verilog files



Boot ROM file:

TLROM.SV

ROM's content is embedded inside the HDL code.

[illegible]

5. Make the system (15/15) Device tree file

Makefile

SBT (.scala)

FIRRTL (.fir)

FPGA (.v)

FPGA (.bit)

Device tree file:

(needed for software)

Chipyard.fpga.arty35t.Arty35
TwoDDRHarness.SmallRocketArt
y35TConfig.dts

Its binary version:

The dtb's content is
embedded inside the TLROM.v

```
L25: cpus {
    #address-cells = <1>;
    #size-cells = <0>;
    timebase-frequency = <1000000>;
    L6: cpu@0 {
        clock-frequency = <0>;
        compatible = "sifive,rocket0", "riscv";
        device_type = "cpu";
        hardware-exec-breakpoint-count = <1>;
        i-cache-block-size = <64>;
        i-cache-sets = <64>;
        i-cache-size = <4096>;
        reg = <0x0>;
        riscv,isa = "rv32imacziCSR_zifencei_zihpm_xrocket";
        riscv,pmpgranularity = <4>;
        riscv,pmpregions = <8>;
        sifive,dtim = <&L5>;
        status = "okay";
        timebase-frequency = <1000000>;
        L3: interrupt-controller {
            #interrupt-cells = <1>;
            compatible = "riscv,cpu-intc";
            interrupt-controller;
        };
    };
};
```

Outline

1. Framework organization
2. SoC Generation
3. FPGA Prototyping
4. Git clone and prepare
5. Make the system
6. **Prepare SDCard**
7. Using IDE
8. Modifying system by Scala config

6. Prepare SDCard (1/6) Prepare SD-card

- Before programming the Arty-A7 FPGA board, prepare the software on the SD card.

- First, you'll need the **gptfdisk** tool to format the SD card.

If you don't have it already, do the following to install it:

From your home folder:

```
$ git clone https://github.com/tmagik/gptfdisk.git  
$ cd gptfdisk/  
$ make -j`nproc`
```

- Put the SD-card into your PC, then:

Go to gptfdisk folder:

```
$ cd gptfdisk/  
$ sudo ./gdisk /dev/sd?
```

The **?** points to the SD card. For example: /dev/sd**b**

6. Prepare SDCard (2/6) Prepare SD-card

- Some commands to use while in the **gptfdisk** tool:

\$ p	: print partitions information
\$ d	: delete partition
\$ n	: create new partition
\$ w	: write partition
\$ q	: exit gptfdisk

- You'll need to format the SD card to look like this:

Number	Start (sector)	End (sector)	Size	Code	Name
1	2048	3071	512.0 KiB	5202	SiFive bare-metal (...)

6. Prepare SDCard (3/6) Prepare SD-card

Example commands:

```
$ sudo ./gdisk /dev/sdb
$ d
$ n
$ (Enter)
$ +1024
$ 5202
$ p
$ w
$ y
```

```
thuc@thuc-Ubuntu:~/temp/gptfdisk$ sudo ./gdisk /dev/sdd
GPT fdisk (gdisk) version 1.0.4

Partition table scan:
  MBR: protective
  BSD: not present
  APM: not present
  GPT: present

Found valid GPT with protective MBR; using GPT.

Command (? for help): d
Using 1

Command (? for help): n
Partition number (1-128, default 1): 
First sector (34-7634910, default = 2048) or {+}size[KMGTP]: 
Last sector (2048-7634910, default = 7634910) or {+}size[KMGTP]: +1024
Current type is 'Linux filesystem'
Hex code or GUID (L to show codes, Enter = 8300): 5202
Changed type of partition to 'SiFive bare-metal (or stage 2 loader)'

Command (? for help): p
Disk /dev/sdd: 7634944 sectors, 3.6 GiB
Model: Multi-Card
Sector size (logical/physical): 512/512 bytes
Disk identifier (GUID): 84456646-2BE9-4A01-8532-37164C9AF21A
Partition table holds up to 128 entries
Main partition table begins at sector 2 and ends at sector 33
First usable sector is 34, last usable sector is 7634910
Partitions will be aligned on 2048-sector boundaries
Total free space is 7633853 sectors (3.6 GiB)



| Number | Start (sector) | End (sector) | Size      | Code | Name                    |
|--------|----------------|--------------|-----------|------|-------------------------|
| 1      | 2048           | 3071         | 512.0 KiB | 5202 | SiFive bare-metal (...) |



Command (? for help): w

Final checks complete. About to write GPT data. THIS WILL OVERWRITE EXISTING
PARTITIONS!!

Do you want to proceed? (Y/N): y
OK; writing new GUID partition table (GPT) to /dev/sdd.
Warning: The kernel is still using the old partition table.
The new table will be used at the next reboot or after you
run partprobe(8) or kpartx(8)
The operation has completed successfully.
thuc@thuc-Ubuntu:~/temp/gptfdisk$
```

6. Prepare SDCard (4/6) Prepare software

- Now, prepare the after-boot software:

From your RISC-V-Demo folder:

```
$ cd RISC-V-Demo/
```

(Note: conda environment must be activated)

Go to:

```
$ cd fpga/sw/
```

Finally, compile the software:

```
$ make
```

6. Prepare SDCard (5/6) Prepare software

- Now, prepare the after-boot software:

From your RISC-V-Demo folder:

```
$ cd RISC-V-Demo/
```

(Note: conda environment must be activated)

Go to:

```
$ cd fpga/sw/
```

Finally, compile the software:

```
$ make
```

- After `$ make`, the compiled software is under the build folder, 3 files need to be considered:

`main.bin` `main.dump` `main.elf`

- **bin:** draw binary file. It will be used to load into SDCard.
- **dump:** assembly file that corresponds to a bin file. It is used to trace when debugging.
- **elf:** binary file like bin. But it includes gdb's debug symbols which are required when debugging with gdb.

6. Prepare SDCard (6/6) Prepare software

- Now, to write the compiled software to the SD card:

From your RISC-V-Demo/fpga/sw folder:

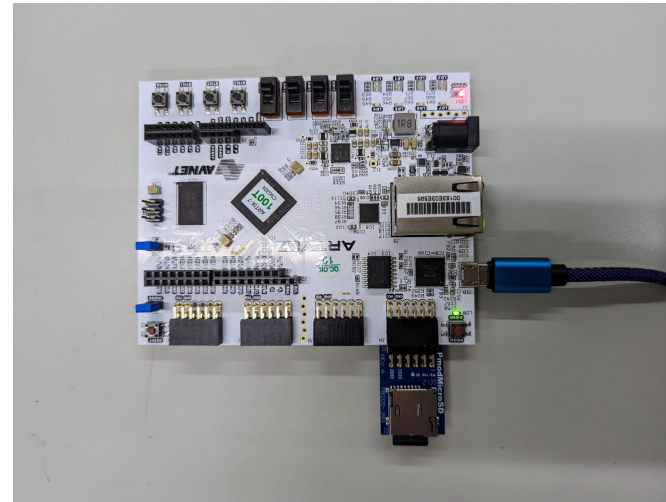
```
$ sudo dd if=./build/main.bin of=/dev/sd?1 conv=fsync bs=4096
```

Again, the **?** points to the SD card.

For example: `$ sudo dd if=./build/main.bin of=/dev/sdb1 conv=fsync bs=4096`

- Then, remove the SD card from your PC.
- Put it in the PMOD-SD-card and connect to the Arty-A7.

Remember to connect the microUSB-to-USB to your PC.



Outline

1. Framework organization
2. SoC Generation
3. FPGA Prototyping
4. Git clone and prepare
5. Make the system
6. Prepare SDCard
- 7. Using IDE**
8. Modifying system by Scala config

7. Using IntelliJ IDEA-IC (1/3) Download and install

To write/modify Scala sources efficiently,
we need an IDE (*Integrated Development Environment*) tool.

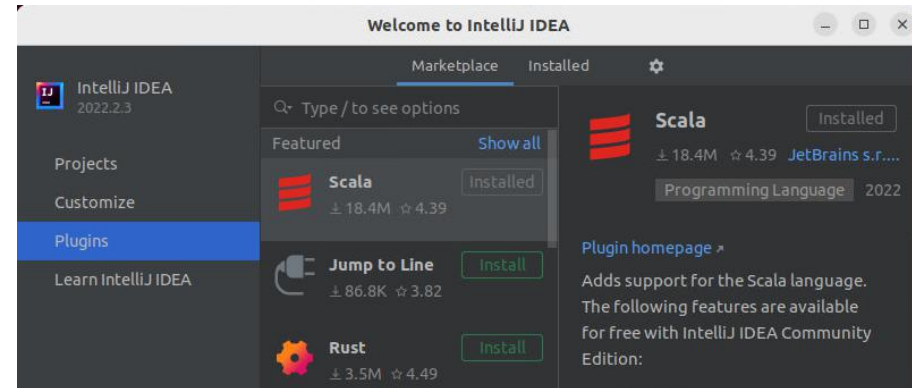
- *SBT* is the compiler for *Scala*
- **IntelliJ IDEA-IC** is the GUI for *SBT*
→ **IntelliJ IDEA-IC** for *Scala* is like Visual Studio for *C++*

To install **IntelliJ IDEA-IC**, just follow their website:

<https://www.jetbrains.com/idea/>

They have a free version: *Community*

The first time it opens, remember to
install the *Scala plugin*.

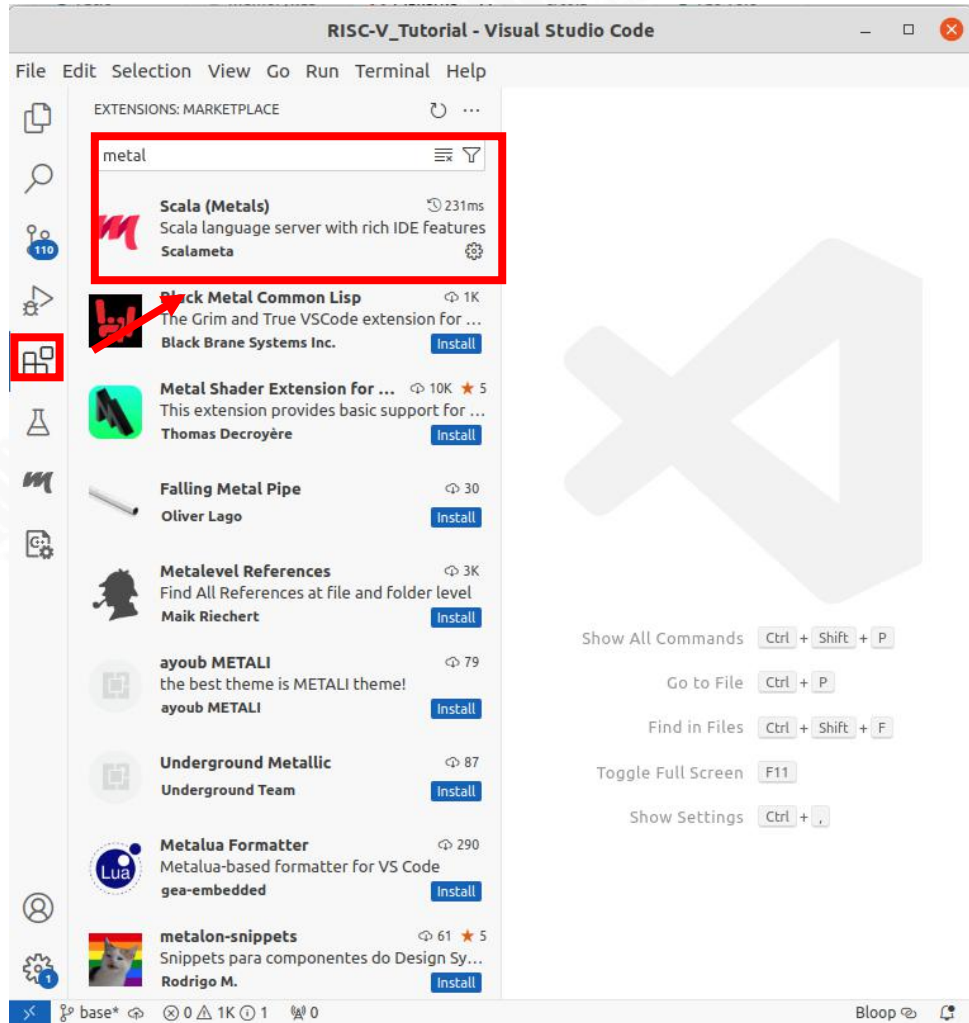


Another option is using Visual Studio Code with Metal package.
Visual Studio Code can be installed directly in Ubuntu:

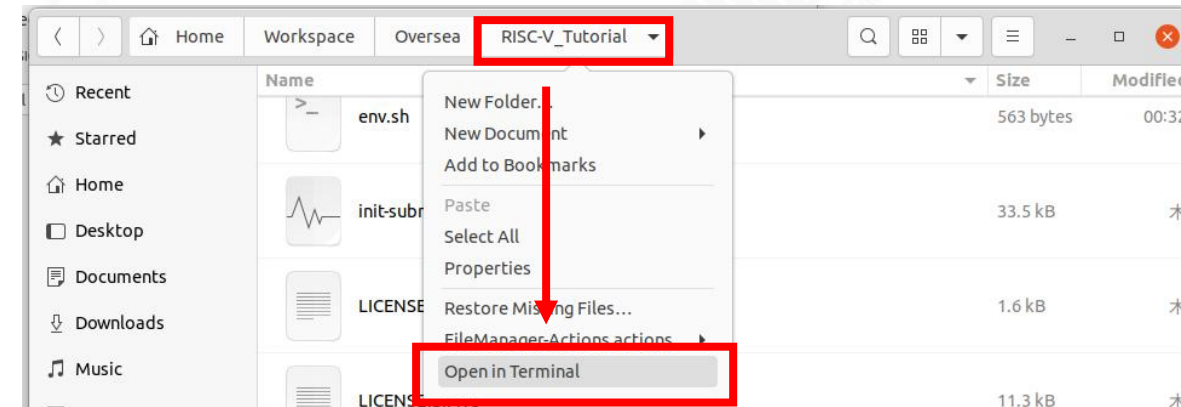
```
$ sudo apt install code
```

7. Work with Visual Studio Code (2/3)

- Install Metal extension: click on Extension icon, and type “metal”. Then select the first appeared extension.

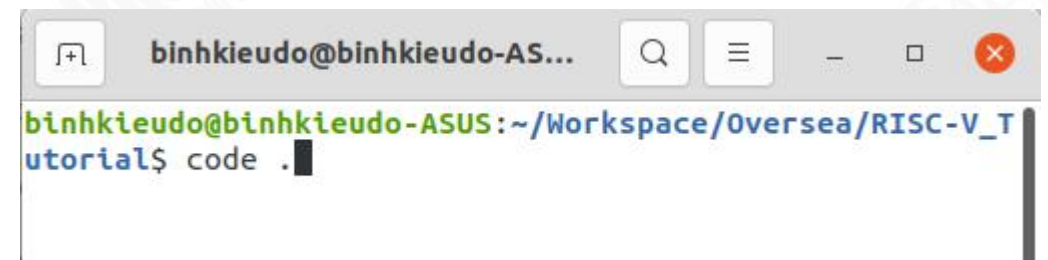


- Open the project in Visual Studio Code by *right click* on the project's folder name. Then *Open in Terminal*

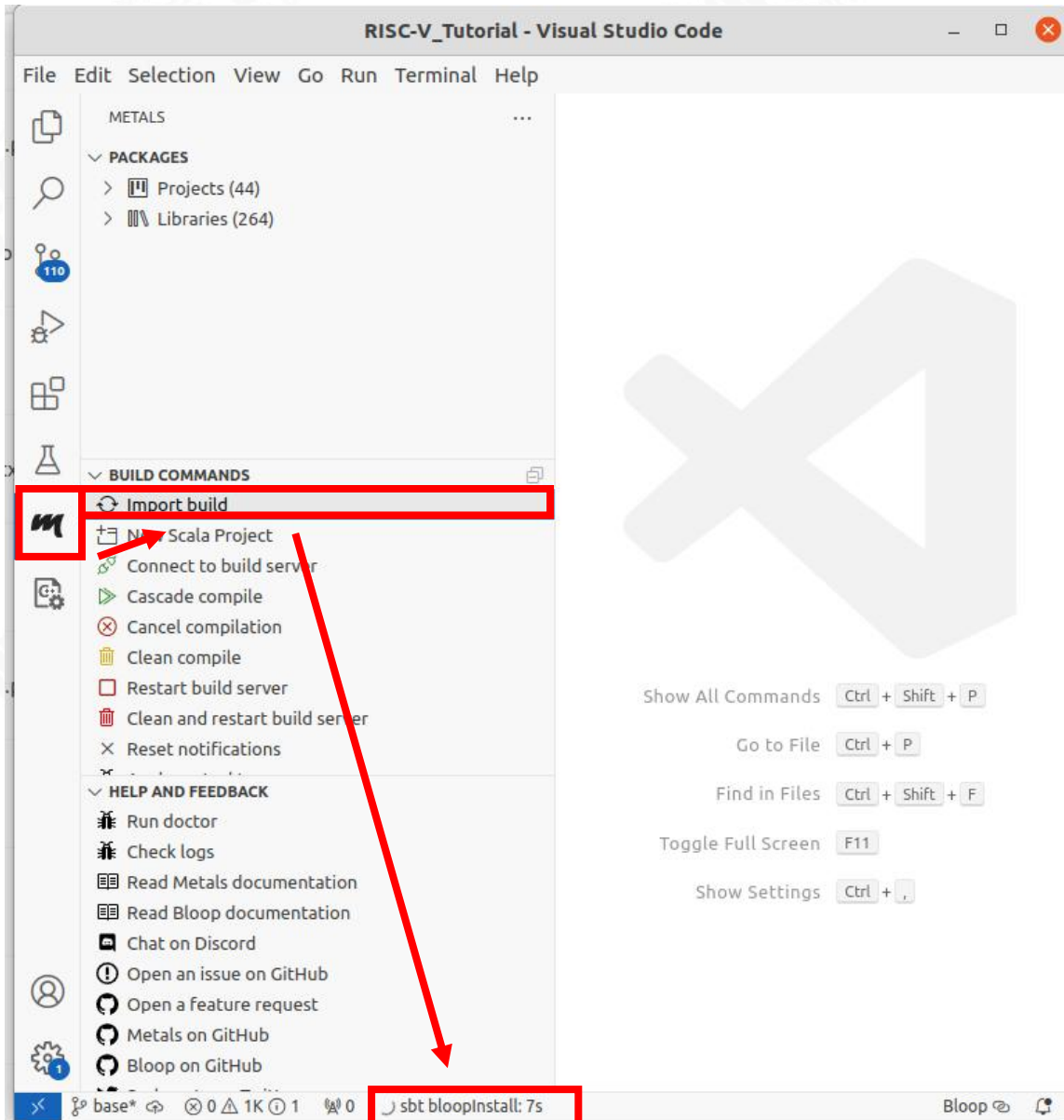


Then type

```
$ code .
```



7. Work with Visual Studio Code (3/3)



In the first use, you need to make Metal scanning the `sbt` project.

1. Click on Metal icon.
2. Click on “Import build”.
3. Wait until “sbt bloopInstall” completed.

Now you can link and highlight the syntax of `scala` and `sbt` files.

Tip: you can install C/C++, Makefile, Verilog/System Verilog Extensions to highlight the syntax of the corresponding files.

Outline

1. Framework organization
2. SoC Generation
3. FPGA Prototyping
4. Git clone and prepare
5. Make the system
6. Program Arty-A7
7. Using IntelliJ IDEA-IC
8. **Modifying system by Scala config**

8. Modifying system by Scala config (1/8) Makefile

The framework allows development separately from the RISC-V Core and deployed platform.

- We begin with the RISC-V core.

Firstly, let's see the Makefile in the FPGA folder.

RISC-V-Demo/fpga/Makefile

RISC-V-
Demo/fpga/src/main/scala/artys35t
/Configs.scala

➔ This class connect the Core Design
with Platform Design

```
70 class SmallRocketArty35TConfig extends Config(  
71   new WithTinyArty35TTweaks ++  
72   new chipyard.config.WithBroadcastManager ++  
73   new chipyard.TinyRocketConfig  
74 )
```

This is the Core Design. This configuration is located at '
RISC-V-

Demo/generators/chipyard/src/main/scala/
config/RocketConfigs/RocketConfigs.scala

Tip: you can jump to this file by right-click on *TinyRocketConfig* then select Go to Definition (or press F12)

```
60 ifeq ($(SUB_PROJECT),arty35t)  
61   # TODO: Fix with Arty  
62   SBT_PROJECT      ?= fpga_platforms  
63   MODEL            ?= Arty35TwoDDRHarness  
64   VLOG_MODEL        ?= Arty35TwoDDRHarness  
65   MODEL_PACKAGE     ?= chipyard.fpga.arty35t  
66   CONFIG            ?= SmallRocketArty35TConfig  
67   CONFIG_PACKAGE     ?= chipyard.fpga.arty35t  
68   GENERATOR_PACKAGE ?= chipyard  
69   TB                 ?= none # unused  
70   TOP                ?= ChipTop  
71   BOARD              ?= arty  
72   FPGA_BRAND         ?= xilinx  
73 endif
```

8. Modifying system by Scala config (2/8) Core modification

RISC-V-

Demo/generators/chipyard/src/main
/scala/config/RocketConfigs/RocketConfigs.scala

```
class TinyRocketConfig extends Config(  
  new chipyard.iobinders.WithDontTouchIOBinders(false) ++  
  new freechips.rocketchip.subsystem.WithIncoherentBusTopology ++  
  new freechips.rocketchip.subsystem.WithNBanks(0) ++  
  new freechips.rocketchip.subsystem.WithNoMemPort ++  
  new freechips.rocketchip.subsystem.With1TinyCore ++  
  new chipyard.config.AbstractConfig)
```

- Go to the definition of `With1TinyCore` class, you will see the configuration of this core. There are some other configurations such as `WithNSmallCores`, `WithNMedCores`, `WithNBigCores`.
- Those are default configurations. You can make your own based on those.



```
class With1TinyCore extends Config((site, here, up) => {  
  case XLen => 32  
  case TilesLocated(InSubsystem) => {  
    val tiny = RocketTileParams(  
      core = RocketCoreParams(  
        useVM = false,  
        fpu = None,  
        mulDiv = Some(MulDivParams(mulUnroll = 8)),  
        btb = None,  
        dcache = Some(DCacheParams(  
          rowBits = site(SystemBusKey).beatBits,  
          nSets = 256, // 16Kb scratchpad  
          nWays = 1,  
          nTLBSets = 1,  
          nTLBWays = 4,  
          nMSHRs = 0,  
          blockBytes = site(CacheBlockBytes),  
          scratch = Some(0x80000000L))),  
        icache = Some(ICacheParams(  
          rowBits = site(SystemBusKey).beatBits,  
          nSets = 64,  
          nWays = 1,  
          nTLBSets = 1,  
          nTLBWays = 4,  
          blockBytes = site(CacheBlockBytes)))  
      )  
    List(RocketTileAttachParams(  
      tiny,  
      RocketCrossingParams(  
        crossingType = SynchronousCrossing(),  
        master = TileMasterPortParams()  
      ))  
    )  
  }  
})
```


8. Modifying system by Scala config (3/8) Core modification

- Aim: add Floating Point Unit (FPU) without Division (div) and Square root (Sqrt) functions. The core is now 32-bit (XLEN = 32) → only 32-bit FPU can be deployed.

```
class With1TinyCore extends Config((site, here, up) => {  
  case XLen => 32  
  case TilesLocated(InSubsystem) => {  
    val tiny = RocketTileParams(  
      core = RocketCoreParams(  
        useVM = false,  
        fpu = None,  
        mulDiv = Some(MulDivParams(mulUnroll = 8))),  
    btb = None,  
    dcache = Some(DCacheParams(  
      rowBits = site(SystemBusKey).beatBits,  
      nSets = 256, // 16Kb scratchpad  
      nWays = 1,  
      nTLBSets = 1,  
      nTLBWays = 4,  
      nMSHRs = 0,  
      blockBytes = site(CacheBlockBytes),  
      scratch = Some(0x80000000L))),  
    icache = Some(ICacheParams(  
      rowBits = site(SystemBusKey).beatBits,  
      nSets = 64,  
      nWays = 1,  
      nTLBSets = 1,  
      nTLBWays = 4,  
      blockBytes = site(CacheBlockBytes)))  
  )  
  List(RocketTileAttachParams(  
    tiny,  
    RocketCrossingParams(  
      crossingType = SynchronousCrossing(),  
      master = TileMasterPortParams()  
    )  
  ))  
})
```

```
class With1TinyCore extends Config((site, here, up) => {  
  case XLen => 32  
  case TilesLocated(InSubsystem) => {  
    val tiny = RocketTileParams(  
      core = RocketCoreParams(  
        useVM = false,  
        fpu = Some(FPUParams(divSqrt=false, fLen=32)),  
        mulDiv = Some(MulDivParams(mulUnroll = 8))),  
    btb = None,  
    dcache = Some(DCacheParams(  
      rowBits = site(SystemBusKey).beatBits,  
      nSets = 256, // 16Kb scratchpad  
      nWays = 1,  
      nTLBSets = 1,  
      nTLBWays = 4,  
      nMSHRs = 0,  
      blockBytes = site(CacheBlockBytes),  
      scratch = Some(0x80000000L))),  
    icache = Some(ICacheParams(  
      rowBits = site(SystemBusKey).beatBits,  
      nSets = 64,  
      nWays = 1,  
      nTLBSets = 1,  
      nTLBWays = 4,  
      blockBytes = site(CacheBlockBytes)))  
  )  
  List(RocketTileAttachParams(  
    tiny,  
    RocketCrossingParams(  
      crossingType = SynchronousCrossing(),  
      master = TileMasterPortParams()  
    )  
  ))  
})
```

8. Modifying system by Scala config (4/8) Core modification

- Return to *RISC-V-Demo/fpga* and *\$ make*, then read *dts* file to see the changes.

Note: we don't need to deploy on board, you can only run *\$ make verilog* to reduce the compile time.

Before

```
L25: cpus {
    #address-cells = <1>;
    #size-cells = <0>;
    timebase-frequency = <10000000>;
    L6: cpu@0 {
        clock-frequency = <0>;
        compatible = "sifive,rocket0", "riscv";
        device_type = "cpu";
        hardware-exec-breakpoint-count = <1>;
        i-cache-block-size = <64>;
        i-cache-sets = <64>;
        i-cache-size = <4096>;
        reg = <0x0>;
        riscv,isa = "rv32imacziocr_zifencei_zihpm_xrocket";
        riscv,pmpgranularity = <4>;
        riscv,pmpregions = <8>;
        sifive,dtim = <&L5>;
        status = "okay";
        timebase-frequency = <10000000>;
        L3: interrupt-controller {
            #interrupt-cells = <1>;
            compatible = "riscv,cpu-intc";
            interrupt-controller;
        };
    };
};
```

F extension
is added



After

```
L25: cpus {
    #address-cells = <1>;
    #size-cells = <0>;
    timebase-frequency = <10000000>;
    L6: cpu@0 {
        clock-frequency = <0>;
        compatible = "sifive,rocket0", "riscv";
        device_type = "cpu";
        hardware-exec-breakpoint-count = <1>;
        i-cache-block-size = <64>;
        i-cache-sets = <64>;
        i-cache-size = <4096>;
        reg = <0x0>;
        riscv,isa = "rv32imafoziocr_zifencei_zihpm_xrocket";
        riscv,pmpgranularity = <4>;
        riscv,pmpregions = <8>;
        sifive,dtim = <&L5>;
        status = "okay";
        timebase-frequency = <10000000>;
        L3: interrupt-controller {
            #interrupt-cells = <1>;
            compatible = "riscv,cpu-intc";
            interrupt-controller;
        };
    };
};
```


8. Modifying system by Scala config (5/8) Platform modification

The framework allows development separately from the RISC-V Core and deployed platform.

- Next, we will modify the platform by changing the address of peripherals.
- Take a look at the serial (UART) and SPI description in `dts` file

Before

```
102 L18: serial@64000000 {
103     clocks = <&L1>;
104     compatible = "sifive,uart0";
105     interrupt-parent = <&L7>;
106     interrupts = <1>;
107     reg = <0x64000000 0x1000>;
108     reg-names = "control";
109 };
110 L20: spi@64001000 {
111     #address-cells = <1>;
112     #size-cells = <0>;
113     clocks = <&L1>;
114     compatible = "sifive,spi0";
115     interrupt-parent = <&L7>;
116     interrupts = <26>;
117     reg = <0x64001000 0x1000>;
118     reg-names = "control";
119 L27: mmc@0 {
120     compatible = "mmc-spi-slot";
121     disable-wp;
122     reg = <0x0>;
123     spi-max-frequency = <1000000>;
124     voltage-ranges = <3300 3300>;
125 };
126
```

We will remove serial.

We will change the address of SPI from 0x64001000 to 0x64000000.

8. Modifying system by Scala config (6/8) Platform modification

- To change the address of SPI from `0x64001000` to `0x64000000`, you only need to replace the corresponding address in `WithDefaultPeripherals` class.

At `RISC-V-Demo/fpga/src/main/scala/arty35t/Configs.scala`

Before

```
class WithDefaultPeripherals extends Config((site, here, up) => {  
  case PeripheryUARTKey => List(UARTParams(address = BigInt(0x64000000L)))  
  case PeripherySPIKey => List(SPIParams(rAddress = BigInt(0x64001000L)))  
  case PeripheryGPIOKey => List(GPIOParams(address = BigInt(0x64002000L), width = 24))  
})
```

```
class WithTinyArty35TTweaks extends Config(  
  // Clock configs  
  new chipyard.harness.WithAllClocksFromHarnessClockInstantiator ++  
  new chipyard.harness.WithHarnessBinderClockFreqMHz(50) ++  
  new chipyard.config.WithMemoryBusFrequency(50.0) ++  
  new chipyard.config.WithSystemBusFrequency(50.0) ++  
  new chipyard.config.WithPeripheryBusFrequency(50.0) ++  
  new chipyard.harness.WithAllClocksFromHarnessClockInstantiator ++  
  new chipyard.clocking.WithPassthroughClockGenerator ++  
  // Harness Binder  
  new WithArty35TUARtharnessBinder ++  
  new WithArty35TSPIISDCardHarnessBinder ++  
  new WithArty35TJTAGHarnessBinder ++  
  new WithArty35TGPIOHarnessBinder ++  
  new WithArty35TTSITieoff ++  
  // IO Binders  
  new WithGPIOIOPassthrough ++  
  new WithUARTIOPassthrough ++  
  new WithSPIIOPassthrough ++  
  new WithTLIOPassthrough ++  
  // Other configurations  
  new WithDefaultPeripherals ++  
  new WithSystemModifications ++  
  new freechips.rocketchip.subsystem.WithoutTLMonitors)
```

After

```
class WithDefaultPeripherals extends Config((site, here, up) => {  
  case PeripheryUARTKey => Nil  
  case PeripherySPIKey => List(SPIParams(rAddress = BigInt(0x64000000L)))  
  case PeripheryGPIOKey => List(GPIOParams(address = BigInt(0x64002000L), width = 24))  
})
```

```
class WithTinyArty35TTweaks extends Config(  
  // Clock configs  
  new chipyard.harness.WithAllClocksFromHarnessClockInstantiator ++  
  new chipyard.harness.WithHarnessBinderClockFreqMHz(50) ++  
  new chipyard.config.WithMemoryBusFrequency(50.0) ++  
  new chipyard.config.WithSystemBusFrequency(50.0) ++  
  new chipyard.config.WithPeripheryBusFrequency(50.0) ++  
  new chipyard.harness.WithAllClocksFromHarnessClockInstantiator ++  
  new chipyard.clocking.WithPassthroughClockGenerator ++  
  // Harness Binder  
  // new WithArty35TUARtharnessBinder ++  
  new WithArty35TSPIISDCardHarnessBinder ++  
  new WithArty35TJTAGHarnessBinder ++  
  new WithArty35TGPIOHarnessBinder ++  
  new WithArty35TTSITieoff ++  
  // IO Binders  
  new WithGPIOIOPassthrough ++  
  // new WithUARTIOPassthrough ++  
  new WithSPIIOPassthrough ++  
  new WithTLIOPassthrough ++  
  // Other configurations  
  new WithDefaultPeripherals ++  
  new WithSystemModifications ++  
  new freechips.rocketchip.subsystem.WithoutTLMonitors)
```


8. Modifying system by Scala config (7/8) Platform modification

- ① Remove the integration of the peripheral in the internal system.
- ② Remove the connections of the pins to the peripheral.

Set the corresponding
key to Nil ①

```
class WithDefaultPeripherals extends Config((site, here, up) => {  
  case PeripheryUARTKey => List(UARTParams(address = BigInt(0x64000000L)))  
  case PeripherySPIKey => List(SPIParams(rAddress = BigInt(0x64001000L)))  
  case PeripheryGPIOKey => List(GPIOParams(address = BigInt(0x64002000L), width = 24))  
})
```

```
class WithDefaultPeripherals extends Config((site, here, up) => {  
  case PeripheryUARTKey => Nil  
  case PeripherySPIKey => List(SPIParams(rAddress = BigInt(0x64000000L)))  
  case PeripheryGPIOKey => List(GPIOParams(address = BigInt(0x64002000L), width = 24))  
})
```

```
class WithTinyArty35TTweaks extends Config(  
  // Clock configs  
  new chipyard.harness.WithAllClocksFromHarnessClockInstantiator ++  
  new chipyard.harness.WithHarnessBinderClockFreqMHz(50) ++  
  new chipyard.config.WithMemoryBusFrequency(50.0) ++  
  new chipyard.config.WithSystemBusFrequency(50.0) ++  
  new chipyard.config.WithPeripheryBusFrequency(50.0) ++  
  new chipyard.harness.WithAllClocksFromHarnessClockInstantiator ++  
  new chipyard.clocking.WithPassthroughClockGenerator ++  
  // Harness Binder  
  new WithArty35TUARTHarnessBinder ++  
  new WithArty35TSPISDCardHarnessBinder ++  
  new WithArty35TJTAGHarnessBinder ++  
  new WithArty35TGPIOHarnessBinder ++  
  new WithArty35TTSITieoff ++  
  // IO Binders  
  new WithGPIOIOPassthrough ++  
  new WithUARTIOPassthrough ++  
  new WithSPIIOPassthrough ++  
  new WithTLIOPassthrough ++  
  // Other configurations  
  new WithDefaultPeripherals ++  
  new WithSystemModifications ++  
  new freechips.rocketchip.subsystem.WithoutTLMonitors)
```

Remove the
connections ②

```
class WithTinyArty35TTweaks extends Config(  
  // Clock configs  
  new chipyard.harness.WithAllClocksFromHarnessClockInstantiator ++  
  new chipyard.harness.WithHarnessBinderClockFreqMHz(50) ++  
  new chipyard.config.WithMemoryBusFrequency(50.0) ++  
  new chipyard.config.WithSystemBusFrequency(50.0) ++  
  new chipyard.config.WithPeripheryBusFrequency(50.0) ++  
  new chipyard.harness.WithAllClocksFromHarnessClockInstantiator ++  
  new chipyard.clocking.WithPassthroughClockGenerator ++  
  // Harness Binder  
  // new WithArty35TUARTHarnessBinder ++  
  new WithArty35TSPISDCardHarnessBinder ++  
  new WithArty35TJTAGHarnessBinder ++  
  new WithArty35TGPIOHarnessBinder ++  
  new WithArty35TTSITieoff ++  
  // IO Binders  
  new WithGPIOIOPassthrough ++  
  // new WithUARTIOPassthrough ++  
  new WithSPIIOPassthrough ++  
  new WithTLIOPassthrough ++  
  // Other configurations  
  new WithDefaultPeripherals ++  
  new WithSystemModifications ++  
  new freechips.rocketchip.subsystem.WithoutTLMonitors)
```

8. Modifying system by Scala config (8/8) Platform modification

- Return to `RISC-V-Demo/fpga` and `$ make`, then read `dts` file to see the changes.
- Note: we don't need to deploy on board, you can only run `$ make verilog` to reduce the compile time.

Before

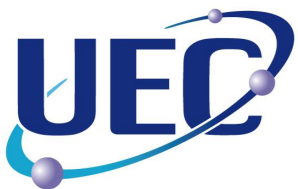
```
102 L18: serial@64000000 {
103     clocks = <&L1>;
104     compatible = "sifive,uart0";
105     interrupt-parent = <&L7>;
106     interrupts = <1>;
107     reg = <0x64000000 0x1000>;
108     reg-names = "control";
109 };
110 L20: spi@64001000 {
111     #address-cells = <1>;
112     #size-cells = <0>;
113     clocks = <&L1>;
114     compatible = "sifive,spi0";
115     interrupt-parent = <&L7>;
116     interrupts = <26>;
117     reg = <0x64001000 0x1000>;
118     reg-names = "control";
119 L27: mmc@0 {
120     compatible = "mmc-spi-slot";
121     disable-wp;
122     reg = <0x0>;
123     spi-max-frequency = <1000000>;
124     voltage-ranges = <3300 3300>;
125 };
126
```

Serial is
removed

Address of
SPI is set to
0x64000000

After

```
91 L15: rom@10000 {
92     compatible = "sifive,rom0";
93     reg = <0x10000 0x10000>;
94     reg-names = "mem";
95 };
96 L18: spi@64000000 {
97     #address-cells = <1>;
98     #size-cells = <0>;
99     clocks = <&L1>;
100     compatible = "sifive,spi0";
101     interrupt-parent = <&L7>;
102     interrupts = <25>;
103     reg = <0x64000000 0x1000>;
104     reg-names = "control";
105 L25: mmc@0 {
106     compatible = "mmc-spi-slot";
107     disable-wp;
108     reg = <0x0>;
109     spi-max-frequency = <1000000>;
110     voltage-ranges = <3300 3300>;
111 };
112
```



国立大学法人

電気通信大学

The University of Electro-Communications

Pham Laboratory
Integrated circuit design laboratory

THANK YOU